

The Civic Ledger

VOL. 1, NO. 015

2026-03-30

COVER STORY

The 10 Skills I Was Missing as a MongoDB User

I spun it up, built my first application, and got a feature working.

MongoDB Blog

When I first started using MongoDB, I didn't have a plan beyond "install it and hope for the best." I had read about how flexible it was, and it felt like all the developers swore by it, so I figured I'd give it a shot. I spun it up, built my first application, and got a feature working.

But I felt like something was missing.

It felt clunky. My queries were longer than I expected, and performance wasn't great; I had the sense that I was fighting with the database instead of working with it. After a few projects like that, I began to wonder if maybe MongoDB wasn't for me.

Looking back now, I can say the problem wasn't MongoDB, but was somewhere between the keyboard and the chair. It was me. I was carrying over habits from years of working with relational databases, expecting the same rules to apply.

If MongoDB's Skill Badges had existed when I started, I think my learning curve would have been a lot shorter. I had to learn many lessons the hard way, but these new badges cover the skills I had to piece together slowly. Instead of pretending I nailed it from day one, here's the honest version of how I learned MongoDB, what tripped me up along the way, and how these Skill Badges would have helped.

Learning to model the MongoDB way

The first thing I got wrong was data modeling. I built my schema like I was still working in SQL— every entity in its own collection, always referencing instead of embedding, and absolutely no data duplication. It felt safe because it was familiar.

Then I hit my first complex query. It required data from various collections, and suddenly, I found myself writing a

series of queries and stitching them together in my code. It worked, but it was a messy process.

When I discovered embedding, it felt like I had found a cheat code. I could put related data together in one single document, query it in one shot, and get better performance.

That's when I made my second mistake. I started embedding everything.

At first, it seemed fine. However, my documents grew huge, updates became slower, and I was duplicating data in ways that created consistency issues. That's when I learned about patterns like Extended References, and more generally, how to choose between embedding and referencing based on access patterns and update frequency.

Later, I ran into more specialized needs, such as pre-computing data, embedding a subset of a large dataset into a parent, and tackling schema versioning. Back then, I learned those patterns by trial and error. Now, they're covered in badges like Relational to Document Model , Schema Design Patterns , and Advanced Schema Patterns .

Fixing what I thought was "just a slow query"

Even after I got better at modeling, performance issues kept popping up. One collection in particular started slowing down as it grew, and I thought, "I know what to do! I'll just add some indexes."

I added them everywhere I thought they might help. Nothing improved.

It turns out indexes only help if they match your query patterns. The order of fields matters, and whether you cover your query shapes will affect performance. Most importantly, just because you can add an index doesn't mean that you should be adding it in the first place.

The big shift for me was learning to read an `explain()` plan and see how MongoDB was actually executing my queries. Once I started matching my indexes to my queries, performance went from “ok” to “blazing fast.”

Around the same time, I stopped doing all my data transformation in application code. Before, I'd pull in raw data and loop through it to filter, group, and calculate. It was slow, verbose, and easy to break. Learning the aggregation framework completely changed that. I could handle the filtering and grouping right in the database, which made my code cleaner and the queries faster.

There was a lot of guesswork in how I created my indexes, but the new Indexing Design Fundamentals covers that now. And when it comes to querying and analyzing data, Fundamentals of Data Transformation is there to help you. Had I had those two skills when I first started, I would've saved a lot of time wasted on trial and error.

Moving from “it works” to “it works reliably”

Early on, my approach to monitoring was simple: wait for something to break, then figure out why. If a performance went down, I'd poke around in logs. If a server stopped responding, I'd turn it off and on again, and hope for the best.

It was stressful, and it meant I was always reacting instead of preventing problems.

When I learned to use MongoDB's monitoring tools properly, that changed. I could track latency, replication lag, and memory usage. I set alerts for unusual query patterns. I started seeing small problems before they turned into outages.

Performance troubleshooting became more methodical as well. Instead of guessing, I measured. Breaking down queries, checking index use, and looking at server metrics side by side. The fixes were faster and more precise.

Reliability was the last piece I got serious about. I used to think a working cluster was a reliable cluster. But reliability also means knowing what happens if a node fails, how quickly failover kicks in, and whether your recovery plan actually works in practice.

Those things you can now learn in the Monitoring Tooling , Performance Tools and Techniques, and Cluster Reliability skill badges. If you are looking at deploying and maintain-

ing MongoDB clusters, these skills will teach you what you need to know to make your deployment more resilient.

Getting curious about what's next

Once my clusters were stable, I stopped firefighting, and my mindset changed. When you trust your data model, your indexes, your aggregations, and your operations, you get to relax. You can then spend that time on what's coming next instead of fixing what's already in production.

For me, that means exploring features I wouldn't have touched earlier, like Atlas Search , gen AI, and Vector Search . Now that the fundamentals are solid, I can experiment without risking stability and bring in new capabilities when a project actually calls for them.

What I'd tell my past self

If I could go back to when I first installed MongoDB, I'd keep it simple:

Focus on data modeling first. A good foundation will save you from most of the problems I ran into.

Once you have that, learn indexing and aggregation pipelines. They will make your life much easier when querying.

Start monitoring from day one. It will save you a lot of trouble in the long run.

Take a moment to educate yourself. You can only learn so much from trial and error. MongoDB offers a myriad of resources and ways to upskill yourself.

Once you have established that base, you can explore more advanced topics and uncover the full potential of MongoDB. Features like Vector Search, full-text search with Atlas Search, or advanced schema design patterns are much easier to adopt when you trust your data model and have confidence in your operational setup.

MongoDB Skill Badges cover all of these areas and more. They are short, practical, and focused on solving real problems you will face as a developer or DBA, and most of them can be taken over your lunch break. You can browse the full catalog at learn.mongodb.com/skills and pick the one that matches the challenge you are facing today. Keep going from there, and you might be surprised how much more you can get out of the database once you have the right skills in place.

{

FEATURES

Generating mock data using Mimesis: Part I

L · Wemake Services

The ability to generate mock but valid data comes in handy in app development, where you need to work with databases. Filling in the database by hand is a time-consuming and tedious process, which can be done in three stages — gathering necessary information, post-processing the data and coding the data generator itself. It gets really complicated when you need to generate not just 10–15 users, but 100–150 thousand users (or other types of data). In this article as well as the two following ones we will introduce you to a tool, which immensely simplifies generating mock data, initial database loading and testing in general.

Mimesis is a Python library, which helps generate mock data for various purposes. The library was written with the use of tools from the standard Python library, and therefore, it doesn't have any side dependencies. Currently the library supports 32 languages and 21 class providers, supplying various data.

A normal way to install mimesis is via pip:

→ pip install mimesis

If for some reason you can not install the package with the help of pip, try installing it manually, as shown below:

```
(venv) → git clone https://github.com/lk-geimfari/mimesis.git (venv) → cd mimesis/ (venv) → python3 setup.py install # or (venv) → make install
```

Please note, that the library only runs on Python 3.3+. Developers don't have any plans of adding Python 2.7 support.

Generating data

Initially, we planned on showing data generation using the example of a small web-application Flask, but we decided against it because not everyone is familiar with Flask nor they are willing to change that. Therefore, we are going to showcase that solely on Python. In case you want to transfer everything to your project on Flask or Django, you simply need to define a static method that will run all the manipulations related to the current model and call it when you need the initial database loading, as demonstrated in the example below.

Model for Flask (Flask-SQLAlchemy) would look like this:

<https://medium.com/media/166d7100b0cdef490e6ae6514550e43a/href>

Now let's transition to shell-mode:

```
(venv) → python3 manage.py shell
```

And generate data. Beforehand, we need to make sure that the database and the model in question are available.

```
>>> db >>> Patient >>> Patient(). _bootstrap(count=40000, locale='en') # generate 40k entries in English.
```

It is worth noting that we will be showing the basic capabilities of the library and we will be using a few most common class providers, since there are too many of them to cover each one in detail. If the article sparks your interest to the library you can visit the useful links listed in the end of the article and find out more.

The library is pretty simple. All you need to do to start working with the data is to create a class provider. The most common type of data in apps are personal users' data, such as name, last name, credit card info, etc. There is a special class provider for this type of data — `Personal()`, which takes the code from the language standard in the form of a line as shown below:

```
>>> from mimesis import Personal >>> person = Personal('is') >>> for _ in range(0, 3): ... person.full_name(gender='male') ... 'Karl Brynjúlfsson' 'Rögnvald Eiðsson' 'Vésteinn Ríkharðsson'
```

Almost every web-application requires e-mail for registration. Naturally, the library supports the ability to generate e-mails with the help of `email()` method `Personal()` class, as below:

```
>>> person.email(gender='female') 'lvana6108@gmail.com' >>> person.email(gender='male') 'john2454@yandex.com'
```

There is a little problem with the method above, which may cause the code to be slightly “dirty” in case the app uses more than one type of class providers. In such situation you should use object `Generic()`, which grants access to all providers from one single object:

```
>>> from mimesis import Generic >>> g = Generic('pl') # pl - code of Poland (ISO 639-1). >>> g.personal.full_name() 'Łonisława Podsiadło' >>> g.datetime.birthday(readable=True) 'Listopad 11, 1997' >>> g.code.imei() '011948003071013' >>> g.food.fruit() 'Cytryna' >>> g.internet.http_method() 'PUT' >>> g.science.math_formula() 'A = (h * (a + b)) / 2'
```

Combining data gives you a vast field for experimentation. For example, you can create mock (female) Visa (Maestro, MasterCard) credit card holders:

```
>>> user = Personal('en') >>> def get_card (sex='female'): ... owner = { ... 'owner': user.full_name(sex), ... 'exp_date': user.credit_card_expiration_date(maximum=21), ... 'number': user.credit_card_number(card_type='visa') ... } ... return owner >>> for _ in range(0, 3): ... get_card() ... {'exp_date': '02/20', 'owner': 'Laverna Morrison', 'card_number': '4920 3598 2121 3328'} {'exp_date': '11/19', 'owner': 'Melany Martinez', 'card_number': '4980 9423 5464 1201'} {'exp_date': '01/19', 'owner': 'Cleora Mcfarland', 'card_number': '4085 8037 5801 9703'}
```

}

{

Agile vs. Waterfall QA: A Comparative Guide

Pankaj Dusane · Helpshift Engineering

Quality Assurance (QA) is a critical aspect of software development, ensuring that the final product meets the desired standards and functions as intended. While QA practices are integral to all development methodologies, the approach to QA can vary significantly depending on whether a project follows the Agile or Waterfall methodology. In this blog post, we'll compare these two methodologies, focusing on their impact on QA processes, workflows, and outcomes.

What is Waterfall QA?

The Waterfall model is a linear and sequential approach to software development. In this methodology, each phase — from requirements gathering to design, development, testing, and deployment — is completed before moving to the next. QA in the Waterfall model is typically conducted during a dedicated testing phase after development is complete.

Key Characteristics of Waterfall QA:

Late Involvement: QA is often involved only after development is finished.

Fixed Scope: Testing is based on a predefined set of requirements and test cases.

Extensive Testing: Since QA occurs after development, it often involves extensive end-to-end testing.

Minimal Flexibility: Changes to requirements or functionality after testing begins can be costly and disruptive.

Predictability: The linear nature of the process makes timelines and deliverables more predictable.

Advantages of Waterfall QA:

Clear documentation and test plans.

Structured processes provide clarity on responsibilities and timelines.

Suitable for projects with well-defined requirements and minimal expected changes.

Challenges of Waterfall QA:

Delayed feedback loops can prolong the discovery of critical issues.

Limited ability to adapt to changes or new requirements.

Potential for larger defects due to the lack of continuous testing.

What is Agile QA?

Agile methodology emphasizes flexibility, collaboration, and iterative development. Unlike Waterfall, Agile integrates QA throughout the development lifecycle, ensuring continuous testing and feedback.

Key Characteristics of Agile QA:

Early Involvement: QA begins during the initial stages of development, such as planning and requirement gathering.

Continuous Testing: QA is performed at every iteration or sprint, often using automated testing tools.

Collaborative Approach: QA teams work closely with developers, product owners, and other stakeholders.

Flexibility: Agile accommodates changing requirements and evolving priorities.

Incremental Delivery: Testing is focused on small, manageable increments of the product.

Advantages of Agile QA:

Faster identification and resolution of defects.

Improved collaboration between teams.

Greater adaptability to changing requirements.

Higher product quality due to continuous testing.

Challenges of Agile QA:

Requires strong communication and collaboration skills.

Frequent testing can increase workload if not automated effectively.

Documentation may be less comprehensive than in Waterfall.

How to transition from Waterfall QA to Agile QA efficiently?

Resistance to Change: Teams habitual to the structured approach of Waterfall may resist Agile practices. Providing clear communication on the benefits of Agile and offering training and workshops would build confidence in the new methodology.

Lack of Agile Expertise: Hiring Agile coaches to guide teams during the transition would help teams struggling to understand Agile principles and practices.

Start Small: Initiate Agile QA practices on a smaller project or a single team before scaling to the entire organization.

Coordination Issues: Agile requires closer collaboration between teams, which can be challenging. Establish regular communication channels, such as daily stand-ups and cross-team meetings.

}

{

The Hidden Layer of Analytics: How QA Builds Trust in Data

Gayatri Panganti · Helpshift Engineering

Every accurate metric is backed by countless validations, events checks and integrity tests in the background.

Quality Assurance in the data-driven systems extends beyond UI validation and backend verification. Such systems rely heavily on data precision and accuracy.

A recent QA focused on validating a productivity analytics framework, ensuring that every event, metric and data flow accurately represented real-world user behaviour. The process was primarily manual, involving live simulations, event validation and detailed metric verification across environment which emphasised logic and data accuracy over automation.

Simulating Real-World Scenarios

Extensive simulations were conducted for multiple user roles as Agent, Supervisor, Admin and Super admin. Each governed by specific dashboard permissions and access rules.

Actions tested included:

Logging in and logging out from the dashboard

Marking presence states such as Available, Online, Or Away

Switching between workzones

Navigating through dashboards

These test conditions generated diverse event streams used to verify how accurately the system captured and processed state transitions.

Parallel Event Validation Through Kafka

Real-time validation was a key aspect of this QA process. Event streams were observed directly through kafka consoles, enabling verification of generated events and their payloads, data structures, and JSON fields as actions were executed.

Each dashboard or SDK action was simulated while the corresponding kafka stream was monitored in parallel, confirming that events triggered correctly and carried accurate data information. Testing covered sandbox, staging, and production environments, ensuring reliability and consistency across all setups.

Data Verification with Metabase and Calculations

After events were processed by the analytics pipeline, metabase queries were used to validate computed metrics against expected outcomes from simulations.

To cross-check results, timestamps (ex. UTC vs IST) were analysed and durations recalculated according to the de-

finer formulas. This combination of query-based data verification with calculated data confirmed the accuracy of each metric and its alignment with underlying event data.

Validation extended to multiple workzones to ensure consistent data aggregation and metric computation globally. The process confirmed that analytics logic produced uniform results regardless of context and configuration.

Edge-Case and Event-Order Testing

The QA process also included detailed validation of edge conditions such as:

Network interruptions during event capture

Out-of-order or missing event sequences

Duplicate triggers or delayed events

Since event order and timing directly affect metric accuracy, verifying event sequence integrity was essential to maintaining trustworthy analytics output.

Understanding and Validating Metric Types

Three core categories of metrics were validated through this effort:

Simple Counter Metrics: Direct counts of user or system actions.

Complex Metrics: Aggregated or state-based computations across multiple events or entities.

Timer Metrics: Duration based calculations between event pairs, used to measure productivity or performance trends.

Each category demanded tailored validation to confirm that event logic, aggregation and computation aligned with business definitions and customer expectations.

Why Accuracy Matters?

Accurate data builds confidence. Every decision informed by analytics relies on the assumption that metrics reflect reality. When analytics are powered by real-time event tracking and computed metrics, even minor errors in the event capture, time-stamping or sequence logic can lead to misleading conclusions.

Through structured simulations, real-time kafka validation and detailed data verification, this QA effort ensured that the analytics system produced metrics users could trust.

QA for analytics systems requires end-to-end validation from event capture to final metric output.

Kafka-based event monitoring provides clear visibility into live data accuracy.

}

{

Using Clojure channels to increase throughput

Abhinav Dubey · Helpshift Engineering

When building systems that process large volumes of messages synchronously, performance bottlenecks can quickly become a challenge specially with single-threaded designs. In this post, we'll look at how leveraging worker threads in a Clojure-based Kafka consumer can significantly boost throughput & reduce total processing time. Using simple concurrency primitives, it's possible to achieve parallelism & scale gracefully, all while keeping the codebase clean & maintainable. We'll start with a baseline, introduce worker threads using Clojure's `core.async` & measure the impact.

Setup & Context

Kafka & Zookeeper

For observability: Grafana

Kafka producer: A simple script that sends messages to a Kafka topic at a configurable rate (messages per minute) for a fixed duration. After each event is pushed, a counter metric is emitted

Kafka consumer: A simple script that listens to a topic & consumes messages & simulates processing time finding square-root of a number (henceforth, assume that it takes 1 second to find the square root) . A counter metric is emitted after processing each message

The Baseline: Single-Threaded Consumer

If each message takes t seconds to process & there are n messages, total processing time becomes $n \times t$ seconds. This provides a clean baseline to evaluate the impact of using channel moving forward.

Adding workers with `core.async`

Values are conveyed on queue-like channels. By default channels require producer and consumer to rendezvous for the transfer of a value through the channel <https://clojuredocs.org/clojure.core.async>

To improve throughput, we introduce parallelism using Clojure's `core.async` channels. Messages from Kafka are fed into a channel, & multiple worker threads read from this channel to process messages concurrently

Here, we used `>!!` (blocking put) & `<!!` (blocking take) to communicate via channels & future to execute the business-logic on a separate thread

Who gets blocked & when :

The thread putting message into the channel will get blocked when there is no space in the buffer

The thread consuming message from the channel will get blocked when there is no message in the buffer

The producer script runs for 5 minutes, producing 200 messages per minute into a topic

Single-threaded consumer: Time taken to process 1k events
16min 30sec

With 3 workers using `core.async` : Time taken to process 1k events
5min 42sec (65% faster than Single-threaded)

With 10 workers using `core.async`: Time taken to process 1k events
5min (70 % faster than Single-threaded & 12% faster than 3 worker-setup)

Please note that we are using just one machine to benchmark the single & multi threaded consumer setup, this might not be ideal in production but is suffice to gauge the impact of adding worker threads on throughput & processing-time.

Kafka consumer : after adding channels The green metric is emitted by kafka-producer. The yellow metric is emitted by the kafka consumer. In the leftmost, we have the single-threaded setup, followed by 3-worker setup & lastly 10-worker setup

Consider another scenario, where the producer script runs for 5 minutes again, but this time it produces 500 messages per minute into the topic

Single-threaded consumer: Time taken to process 2.5k events
40 min 28sec

With 3 workers threads: Time taken to process 2.5k events
13min 39sec (66% faster than Single-threaded)

With 10 workers threads: Time taken to process 2.5k events
5min 6sec (87 % faster than Single-threaded & 62% faster than 3 worker-setup)

Comparing load averages

Comparing 1 minute & 5 minute load-average of single-threaded & 10-worker thread consumer

One of the primary trade-off between single-threaded & multi-threaded consumers lies in balancing system load with processing throughput.

While multi-threaded consumers provide higher parallelism & faster event handling, they also lead to increased CPU usage & a noticeably higher system load average. On the other hand, single-threaded consumers maintain a lower & more predictable system load but process events at a slower rate.

During the benchmarking, it was observed that transitioning to a multi-threaded model caused the load average to rise significantly. This increase — measured using the `uptime` command — highlights the computational overhead introduced by concurrency & highlights the need to tune thread count & workload based on the system's capacity

}

{

MongoDB.local NYC 2025: Definindo o Banco de Dados Ideal para a Era da IA

MongoDB Blog

Ontem, demos as boas-vindas a milhares de desenvolvedores e executivos no MongoDB.local NYC, a mais recente parada do nosso .local séries. Ao longo do último ano, conectamos-nos com dezenas de milhares de parceiros e clientes em 20 cidades ao redor do mundo. Mas é especialmente significativo estar em Nova York, onde o MongoDB foi fundado e onde ainda temos nossa sede.

Durante o evento, apresentamos novas capacidades que reforçam a posição do MongoDB como o principal banco de dados moderno do mundo. Com o MongoDB 8.2, nossa versão mais rica em recursos e de melhor desempenho até agora, estamos elevando o padrão do que os desenvolvedores podem alcançar. Também compartilhamos mais sobre nossos modelos de incorporação e reclassificação da Voyage AI , que trazem precisão e eficiência de última geração para a criação de aplicativos de IA confiáveis e seguros. E com a pesquisa e a pesquisa vetorial agora em visualização pública para o MongoDB Community Edition e o Enterprise Server , estamos colocando poderosas funcionalidades de recuperação diretamente nos ambientes dos clientes—onde quer que eles prefiram executar.

Estou particularmente entusiasmado com o lançamento da Plataforma de Modernização de Aplicativos MongoDB , ou AMP. Empresas em todo o mundo estão lidando com os custos massivos dos sistemas legados que não conseguem dar suporte às demandas da IA. AMP não é um simples “lift-and-shift”. É uma plataforma repetível e de ponta a ponta que combina ferramentas impulsionadas por IA, técnicas comprovadas e talento especializado para reinventar sistemas críticos de negócios, minimizando custos e riscos. Os primeiros resultados são impressionantes: as empresas que estão migrando de sistemas antigos para o MongoDB estão fazendo isso de duas a três vezes mais rápido, e tarefas como a reescrita de código estão acelerando em uma ordem de magnitude.

Figura 1. Palestra principal do MongoDB.local NYC.

Assista à palestra completa no YouTube .

Tornar-se o banco de dados moderno mais popular do mundo.

Ao refletir sobre a jornada do MongoDB, fico impressionado com o quanto avançamos. Quando entrei há pouco mais de uma década, tínhamos apenas alguns milhares de clientes. Hoje, o MongoDB serve a quase 60.000 organizações em todos os setores e verticais, incluindo mais de 70% das empresas da Fortune 100 e inicializações nativas de IA de ponta.

No entanto, a razão por trás do nosso crescimento permanece a mesma. Os relational database criados na década de 1970 nunca foram projetados para dimensionar a escala e complexidade dos aplicativos modernos. Eles eram rígidos, difíceis de dimensionar e lentos para se adaptar. Nossos

fundadores, que vivenciaram essas limitações em primeira mão ao construir a DoubleClick, decidiram criar algo melhor: um modelo de banco de dados projetado para as realidades do mundo moderno. O modelo de documento foi criado.

Baseado em JSON, o modelo de documento é intuitivo, flexível e poderoso. Ele permite que os desenvolvedores representem dados complexos, interdependentes e em constante mudança de maneira natural. E, ao entrarmos na era da IA, essas mesmas qualidades — adaptabilidade, escalabilidade e segurança — são mais críticas do que nunca. O banco de dados que uma empresa escolhe será uma das decisões mais estratégicas que determinarão o sucesso de suas iniciativas de IA.

Os aplicativos de IA generativa já começaram a oferecer ganhos de produtividade, escrever código, elaborar documentos e responder a perguntas. Mas a verdadeira transformação está por vir com a IA agêntica — aplicativos que percebem, decidem e agem. Esses agentes inteligentes não apenas seguem fluxos de trabalho; eles buscam resultados, raciocinando sobre as melhores etapas para alcançá-los. E nesse loop, o banco de dados é indispensável. Ela fornece a memória que permite aos agentes perceberem o contexto, os fatos que lhes permitem decidir de forma inteligente e o estado que lhes permitirá agir de forma coerente.

É por isso que os dados de uma empresa são seu ativo mais valioso. LLM (LLMs) podem gerar respostas, mas é o banco de dados que fornece continuidade, colaboração e verdadeira inteligência. O futuro da IA não é apenas sobre raciocínio—é sobre contexto, memória e o poder dos seus dados.

O banco de dados ideal para IA transformativa

Então, como é o banco de dados ideal para IA agêntica? Deve refletir a complexidade de hoje e as mudanças de amanhã. Deve falar a linguagem da IA, que é cada vez mais JSON. Ele deve integrar a recuperação avançada de dados brutos, metadados e embeddings — não apenas a correspondência exata, mas também o significado e a intenção. Ele deve conectar dados privados e LLM com incorporações e reclassificadores da mais alta qualidade. E deve fornecer o desempenho, a escalabilidade e a segurança necessários para suportar aplicativos de missão crítica em um dimensionamento global.

Isso é precisamente o que o MongoDB entrega. Nós não apenas marcamos as caixas desta lista — nós as definimos.

Estamos apenas dando os primeiros passos

É por isso que estou tão otimista quanto ao nosso futuro. A energia e a criatividade que vemos em todos os eventos do MongoDB.local me lembram da paixão que sempre impulsionou esta empresa. À medida que nossos clientes continuam a inovar, sei que o MongoDB está na posição ideal para ajudá-los a prosperar na era da IA.

}

{

Dependency Injection, MVP, and the all reliable Android Studio IDE.

Aleksei Bingham · Strava Engineering

First and foremost, thanks for visiting my little slice of the internet! I hope the following is both insightful and comedic as I plan to @inject well constructed jokes throughout this post.

Hello! My name is Aleksei and at the time of writing this I'm a rising Senior perusing a Bachelors of Science in Computer Science at the Rochester Institute of Technology. Specifically, I'm focusing on Programming Languages and Tools, a cluster of courses offered by the Golisano College of Computing and Information Sciences at RIT. Clusters are courses that supplement the core Computer Science curriculum allowing students to dive deeper into the topics that spark their interests the most. Below you will find the courses I have and will be taking from this cluster.

CSCI-742 Compiler Construction (Completed)

CSCI-541 Programming Skills in Haskell (Fall 2022)

CSCI-541 Systems Programming in Rust (Spring 2023)

As you can probably tell, I have a love for languages. But don't be mistaken, computer programming languages are not the only type of languages I enjoy studying. While at RIT I also studied Russian, the language that was almost my native tongue. While my future schedule doesn't support taking more Russian courses I have continued to study the language on my own time.

Outside of academics, I'm a passionate endurance athlete who trains to compete throughout three seasons: cross country, indoor, and outdoor track & field. I primarily race the 3K, 5K, and 8K and plan to continue my love for racing outside of college out on the roads running the 15K, half, and the marathon.

Enough about me, let's switch over to talking about the heart of this blog post. The reason why I decided to spend a summer at Strava.

Why Strava?

It goes without saying, I love to run. I competed in cross-country, indoor, and outdoor track and field all of high-school and planned to do the same in college. Surprisingly, I didn't use Strava throughout high-school. In fact, I don't think I knew of Strava's existence in the first place. It wasn't until I arrived on campus that my new teammates would bug me to install Strava so I could join the team group. It was from that moment on that I was hooked.

There was something special about seeing my teammates's activities on Strava. Whether I was finishing up a brutal workout or just jogging around the neighborhood I always had something to share on Strava and so did my teammates. Strava provided a feeling of camaraderie and I knew I wanted to be apart of it.

I vividly remember reading other intern blog posts from previous years. The team activities, company mission, and passion for the product showed through each and every medium post I stumbled across. It was for that reason that I had dreamed of landing an internship at Strava before graduation and ultimately would apply as an Android Engineer Intern during the Fall of 2021. In fact, I still have the activities leading up to my offer letter on Strava!

With a couple of weeks until the start of my internship I learned that I would be placed on the growth team. The growth team is all about experimentation in which product managers, designers, and engineers collaborate to drive user retention and subscription purchases. Specifically, I was going to be working on the subscription acquisition team inside growth where the primary goal is to encourage users to subscribe for all of Strava's awesome features.

Throughout my first week I spent the majority of my time learning Kotlin, Android Studio, and basic Android Development all the while meeting my new teammates. With the help of my remarkable mentor, Faraz , I was able to hack together the following path of execution:

Unfortunately, I only had a single thread to work with and many of these procedures were blocking operations

@Target(Dagger. Dependency, RxJava. Dependency and MVP. Design_Pattern)

so needless to say, learning all of this happened over the course of my internship.

While I don't have the time (or energy) to write about everything I did throughout this internship. I will be highlighting the project that I worked on in significant detail.

Shortly after my ramp-up period my manager disclosed my summer project with me. I had been assigned to bring student plans to Android. Previously, student plans was only accessible on our website . However, a significant portion of our athletes are also accessing Strava through our iOS and Android apps. Without including student plans on mobile we are potentially missing out on thousands if not millions of athletes.

At first glance, the project seemed trivial with just a few UI changes and a single backend API call. Keep in mind, my previous Android experience is practically nonexistent despite having published some (horrendous) apps that I am too embarrassed to share here. Needless to say I was feeling quite confident.

After a design review with our team's phenomenal UI/UX designer (huge shoutout to Grace Kim. I will never understand how you come up with these designs) I was given a portion of the overall design and this is what it looked like:

Prior to seeing the code firsthand, I had assumed the settings view would be a simple activity with a vertical linear layout using a reusable custom component responsible for

}

{

Form Validation Tips Every Web Developer Should Know!

Hritik Jaiswal · Helpshift Engineering

Forms are everywhere online, from signing up for newsletters to making purchases. But let's be honest — nothing's more frustrating than a form that's hard to fill out or riddled with unclear error messages. In this post, we'll dive into practical tips and tricks to make your form validation seamless, user-friendly, and maybe even enjoyable!

We'll walk through tips for using built-in HTML features and creating custom validation with JavaScript. No complicated jargon — just practical steps to improve your forms.

Tip 01: Use the Correct HTML Input Types

Before applying correct HTML Input Types

If you set the input type as “text” for a password field, the password will not be obscured as you type. Similarly, if you use the “text” input type for an email field, the browser's default email pattern check will not be triggered.

When you use the correct input type, the password field will obscure the characters being typed, which is the desired behavior. Additionally, if you type an incorrect email address in the email field, the browser will notify you with a pop-up because of the built-in validation it provides.

Email should use type=“email”.

Password should use type=“password”.

Confirm Password should also be used type=“password”.

After applying correct HTML Input Types

Code :

Add Form Validation Tip #1 · [hritik5102/Form-Validation-Tips@502e416](#)

Tip 02: Built-in HTML Validators

If the username is the required field in your form. To ensure that the username field contains valid data, mark it as “required.” If the user tries to submit the form without filling in this field, the browser will prompt them to complete it. You can also restrict the number of characters by using the “minlength” attribute.

For Username, mark it as “required.” If empty upon submission, the browser will ask: “Please fill out this field.”

Use minlength=“5” to require a minimum of five characters.

Tip 02: Built-in HTML Validators

Code :

Add Form Validation Tip #2 · [hritik5102/Form-Validation-Tips@cd8abbb](#)

Tip 03: Using Regex for Pattern Matching

For instance, to require a number in the username, use the pattern attribute: pattern=“^[0-9]+\$”.

If a string is entered instead of a number, an HTML tooltip will appear, prompting: “Please match the requested format.”

Tip 03: Using Regex for Pattern Matching

Code :

Add Form Validation Tip #3 · [hritik5102/Form-Validation-Tips@3b3230b](#)

Tip 04: Custom Tooltip Validation Messages

The tooltip message “Please match the requested format” doesn't clearly explain the specific format we expect from the user.

We can use the title attribute to display custom tooltips. For example, on the username field, set title=“Username must be at least 5 characters and include a number.”

Tip 04: Custom Tooltip Validation Messages

Code :

Add Form Validation Tip #4 · [hritik5102/Form-Validation-Tips@9e64865](#)

Tip 05: Create Custom Tooltip Messages with JavaScript

reportValidity() is a method available on form elements (, , , etc.) that triggers the browser's built-in form validation.

setCustomValidity(message) is a method on input elements that lets you set a custom validation message, overriding the browser's default message for specific constraints.

Together, they provide a robust mechanism for client-side form validation in web applications.

Tip 05: Create Custom Tooltip Messages with JavaScript

Code :

Tip 06: Avoid Early Validation using onChange event handler Notice how the error message is displayed only when the end user shifts focus to the next input field.

Hritik Jaiswal

Add Form Validation Tip #5 · [hritik5102/Form-Validation-Tips@73c63bc](#)

}

{

Explore, expand, exploit

Mary Rose Cook

A few months ago, I started sleeping badly.

I had been excited about AI since ChatGPT came out. I'd loved using Cursor to help me program since Jay had told me about it over the phone as I walked from Eureka Heights back home to Noe Valley.

But, in January, something changed. The proximate cause was a flood of new AI releases. o3-mini, Deep Research, Lightpage. Every week, more intelligence dropping from heaven into my lap.

But the bigger change was that I was getting more productive, faster.

Type in a few sentences, get a hundred lines of code. A feeling of vertigo.

More than that, I could learn a new technique in an hour and become significantly more productive.

This was in stark contrast to the previous twenty years I'd spent learning to program. That was a slow, accretive grind. A new technique for encapsulation. A more refined understanding of what it means to "repeat yourself". Learning that you could step-debug a production web app.

My friend, Sam, has this model of learning as building a graph. Each node is a piece of information or a skill or a behavior. They're interconnected. Acquiring a new node of knowledge isn't too hard. It's a bit harder to elaborate it. Which is to say, to connect it to the existing nodes in your graph.

But the real fucker is when you have to unmake a part of your graph. You get cognitive dissonance because some of the nodes contradict each other or need to be pried apart or replaced. It's very painful to disassemble the graph and remake it. Learning to program was a lot of that.

Learning to build software with AI feels completely different. It's much closer to learning a new discipline. Certainly, the old way of programming is relevant. But all the power comes from the new techniques in this new field that doesn't even really have a name.

Further, a lot of the new techniques involve a new workflow. Copy code from your editor into GPT, make a request, get code back, paste it into your editor. No, don't do that any more. Instead, start by selecting code and then pressing ⌘-L. No, wait, stop. Just press ⌘-L, make a request, get code, press the Apply button. No. New move. Press ⌘-I, make a request, scan the code as it's added to the repo, run and check the behavior. No, wait, this is the killer. Iterate on a PRD first, then tell the LLM to write the code in one shot.

}

Adopting a new workflow induces cognitive strain because it requires extra supervision. And it requires willpower to not just do things the old safe way. It's exhausting.

So the destabilization is three things. First, the pure sensation of doing something with a new, startling ease. Second, the knowledge that I can put in ten minutes or an hour or two and get significantly faster at building software. Third, my methods and working life are changing every day.

With gains like that to be had, why wouldn't I do it all the time? Stay up later. Get up earlier. Watch AI programming game tape with lunch. Spend an hour learning rather than working, then make up the loss in the same day.

There's also a problem. There is so much information. Every day is a deluge of new tools, techniques and streams. Which ones are worthwhile? I have a long list of stuff I've been meaning to try, that seems promising, that worked and I feel I should be doing more. Sometimes I'll watch a forty-five minute YouTube video of someone vibe coding and it'll be useless. Other times, I'll skip forward and hit a nugget so juicy that I'll become terrified I'm missing these sorts of things all over the place. Sometimes a technique will seem unpromising but, several hours in, will click.

For this, at least, I've found something of a solution. When you're in a fast-changing environment with high uncertainty, what do you do? Well, Civilization, the video game, is a good guide. You explore, expand, exploit.

You spend a good deal of time exploring widely. Things are changing rapidly and there's a lot to learn. Many explorations will be fruitless and that's totally fine.

When you find something good, you expand your technique to incorporate it. Did you hear about an obviously useful tool? Install it! Have you been refactoring your code to make it easier to change? Do it!

And when something's working, you exploit it as much as possible. It's easy to learn a cool new technique then forget to use it. Don't. Instead, bring it back.

I keep a list with these three categories. Explore, expand, exploit. And I spend some time on each.

It's something to cling to.

{

Headerless train announcements

Oona Räisänen · Oona Raisanen (*windytan*)

The Finnish state railway company just changed their automatic announcement voice, discarding old recordings from trains. It's a good time for some data dumpster diving for the old ones, don't you think?

A 67-megabyte ISO 9660 image is produced that once belonged to an older-type onboard announcement device. It contains a file system of 58 directories with five-digit names, and one called "yleis" (Finnish for "general").

Each directory contains files with three-digit file names. For each number, there's 001.inf, 001.txt and 001.snd. The .inf and .txt files seem to contain parts of announcements as ISO 8859 encoded strings, such as "InterCity train" and "to Helsinki". The .snd files obviously contain the corresponding audio announcements. There's a total of 1950 sound files.

Directory structure

The file system seems to be structurally pointless; there's nothing apparent that differentiates all files in /00104 from files in /00105. Announcements in different languages are numerically separated, though (/001xx = Finnish, /002xx = Swedish, /003xx = English). Track numbers and time read-outs are stored sequentially, but there are out-of-place announcements and test files in between. The logic connecting numbers to their meanings is probably programmed into the device for every train route.

Everything can be spliced together from almost single words. But many common announcements are also recorded as whole sentences, probably to make them sound more natural.

Audio format

The audio files are headerless; there is no explicit information about the format, sample rate or sample size anywhere.

The byte histogram and Poincaré plot of the raw data suggest a 4-bit sample size; this, along with the fact that all files start with 0x80, is indicative of an adaptive differential PCM encoding scheme.

Unfortunately there are as many variations to ADPCM as there are manufacturers of encoder chips. None of the decoders known by SoX produce clean results. But with the right settings for the OKI-ADPCM decoder we can already hear some garbled speech under heavy Brownian noise.

For unknown reasons, the output signal from SoX is spectrum-inverted. Luckily it's trivial to fix (see my previous post on frequency inversion). The pitch sounds roughly natural when a 19,000 Hz sampling rate is assumed. A test tone found in one file comes out as a 1000 Hz sine when the sampling rate is further refined to 18,930 Hz.

This is what we get after frequency inversion, spectral equalization, and low-pass filtering:

There's still a high noise floor due to the mismatch between OKI-ADPCM and the unknown algorithm used by the announcement device, but it's starting to sound alright!

There seems to be an announcement for every thinkable situation, such as:

"Ladies and Gentlemen, as due to heavy snowfall, we are running slightly late. Please accept our apologies."

"Ladies and Gentlemen, an animal has been run over by the train. We have to wait a while before continuing the journey."

"Ladies and Gentlemen, the arrival track of the train having been changed, the platform is on your left hand side."

"Ladies and Gentlemen, we regret to inform you that today the restaurant-car is exceptionally closed."

Also, there is an English recording of most announcements, even though only Finnish and Swedish are usually heard on commuter trains.

One file contains a long instrumental country song.

In an eerily out-of-place sound file, a small child reads out a list of numbers.

Final words

This is something I've wanted to do with this almost melodically intonated announcement about ticket selling compartments.

But with the right settings for the OKI-ADPCM decoder we can already hear some garbled speech under heavy Brownian noise.

Oona Räisänen

}

{

Começando a destacar a excelência: MongoDB GlobalPartner Services 2025

MongoDB Blog

Em um mundo que está sendo remodelado pela IA e pelas rápidas mudanças tecnológicas, uma coisa é clara: nossos parceiros estão impulsionando o futuro com o MongoDB. Juntos, ajudamos os clientes a modernizar sistemas legados, resolver desafios que vão desde segurança até restrições orçamentárias, e criar a próxima onda de aplicações com tecnologia de IA.

É por isso que temos orgulho de fazer o anúncio do MongoDB Global Partner Awards anual — celebrando os parceiros que lideraram o caminho em 2025. Desde a liderança em IA e modernização até o avanço da inovação no setor público e a construção de colaborações ousadas para o mercado, esses parceiros estabelecem o padrão de excelência. A liderança deles não apenas move a agulha — ela redefine o que é possível.

Parceiro Global de Cloud do Ano: Microsoft

Temos o orgulho de reconhecer a Microsoft pelo excepcional crescimento ano a ano como parceiro Global de Cloud do ano do MongoDB. Juntas, MongoDB e Microsoft têm impulsionado fortemente setores como saúde, telecomunicações e serviços financeiros, ajudando organizações a desenvolver aplicações excepcionais que oferecem experiências incríveis aos clientes.

O profundo compromisso da Microsoft com a colaboração, o sucesso do cliente e a liderança em nuvem a torna uma parte indispensável do ecossistema de parceiros do MongoDB. A força da parceria continua a crescer; na verdade, o MongoDB foi recentemente selecionado como parceiro da Microsoft para um “Unify your data solution play”, que permite aos clientes aproveitar as integrações conjuntas e os recursos de go-to-market (GTM) entre o MongoDB Atlas no Azure e os serviços nativos da Microsoft.

Parceiro global de nuvem de IA do ano: Amazon Web Services (AWS)

A AWS tem sido uma força motriz em ajudar os clientes a desbloquear todo o potencial da IA com o MongoDB, destacando-se pelo nosso trabalho com a Novo Nordisk, que utilizou o Amazon Bedrock e o MongoDB Atlas para desenvolver uma solução de IA que reduziu um de seus fluxos de trabalho mais demorados de 12 semanas para 10 minutos. O trabalho com a Novo Nordisk é apenas um exemplo de muitos que demonstra o poder da nossa parceria para criar diferenciação de negócios para clientes na era da IA generativa.

O MongoDB também foi parceiro de lançamento da Competência em IA Generativa para a AWS, estreitando ainda mais nossa colaboração em IA. Desde casos de uso revolucionários de IA generativa e além, nossa parceria capacita as organizações a se moverem mais rapidamente, inovarem com mais ousadia e se transformarem com confiança. Jun-

tos, AWS e MongoDB estão moldando o que é possível na era da IA.

Parceiro GTM global do ano em nuvem: Google Cloud

O Google Cloud está sendo homenageado por acelerar novos negócios por meio de iniciativas conjuntas de GTM impactantes. A parceria do MongoDB com o Google Cloud estabeleceu o padrão para uma colaboração significativa —impulsionando novos negócios e gerando impacto em algumas das empresas globais mais complexas do mundo. O programa conjunto de Representantes de Desenvolvimento de Vendas do Google Cloud e do MongoDB tem sido a pedra angular desse sucesso, garantindo que talentos em estágio inicial tenham a oportunidade de trabalhar com as maiores organizações do mundo enquanto aprendem um manual de vendas que lhes servirá bem pelo resto de suas carreiras. O Google Cloud continua a ser uma força impulsionadora no crescimento global do MongoDB graças ao seu compromisso conjunto com estratégias inovadoras de GTM.

Parceiro Global de Integração de Sistemas do Ano: Accenture

A Accenture demonstrou um compromisso excepcional como Parceiro Global de SI, estabelecendo um centro de excelência dedicado para o MongoDB dentro de sua linha de serviços de engenharia de software.

Juntos, MongoDB e Accenture entregaram resultados transformadores para clientes em diversos setores, desde a modernização de pagamentos para um banco líder até a transformação de dados para um grande fabricante. Enquanto isso, a colaboração mais estreita com a unidade de negócios BFSI da Accenture continuou a impulsionar o sucesso global dos clientes. Ao combinar a moderna plataforma MongoDB de banco de dados com a profunda experiência no setor da Accenture, nossa parceria continua a ajudar os clientes a se modernizarem, desbloquear perspicácia baseada em dados e acelerar a transformação digital em escala empresarial.

Parceiro Global do Setor Público do Ano: Accenture Federal Services

A Accenture Federal Services desempenhou uma função fundamental no avanço da presença do MongoDB no setor público. Graças ao seu dimensionar, experiência e foco nos resultados do cliente, ela impulsionou um crescimento notável ano após ano e tem apoiado missões críticas do governo em coordenação com o MongoDB.

No último ano, MongoDB e Confluent fortaleceram o alinhamento global de go-to-market (GTM), focando na aceleração do engajamento de co-venda na EMEA e APAC.

}

{

Carrying Complexity, Delivering Agility

MongoDB Blog

Resilience, intelligence, and simplicity: The pillars of MongoDB's engineering vision for innovating at scale

We're relatively new to MongoDB—Ashish joined two years ago via the Granite acquisition after a decade-plus building Google's databases and distributed systems, and Akshat joined in June 2024 after 15 years building databases at AWS. We have a shared obsession with distributed systems. We'd seen how much developers loved MongoDB, which is part of the reason we joined the company—MongoDB is one of the most loved databases in the world. So one of the first things we sought to understand was why.

It turned out to be simpler than we thought: MongoDB's vision is to get developers to production fast. This means making it easy to start, and easier to keep going—one command spin-up, sane defaults for day one, and zero downtime upgrades and zero downtime expansion to multiple clouds as you scale. That's what developer agility looks like in practice: the ability to choose the best tools, move quickly, and to trust the system to carry the weight of failure, complexity, and change.

At MongoDB, three principles drive that vision: resilience, intelligence, and simplicity.

Resilience is the ability to keep going when something breaks, intelligence is the ability to adapt to changing conditions, and simplicity is reducing cognitive and operational load so users and operators can move quickly and safely. These are not just technical goals—we treat them as non-negotiable design constraints. So if a change widens blast radius, breaks adaptive performance, or adds operator toil, it doesn't ship.

In this post, we share the key engineering themes shaping our work and the mechanisms that keep us honest.

Security as a first principle

Security isn't a wall you build around your data. It's an assumption you design against from the very beginning. The assumption is simple: in a distributed system, you can't trust the network, you can't trust the hardware, and you certainly can't trust your neighbors.

This starts with architectural isolation. In most cloud database service offerings, you're sharing walls with strangers. Shared walls hurt performance, they leak failures, and sometimes they leak secrets. We minimize shared walls, and where utilities must be shared, we build firebreaks. Stronger isolation reduces the blast radius of mistakes and attacks.

With a MongoDB Atlas dedicated cluster, you get the whole building. Your cluster runs on its own provisioned servers, in its own private network (VPC). Your unencrypted data is never available in a shared VM or process. There are no

"noisy neighbors" because you have no neighbors. The attack surface shrinks dramatically, and resource contention disappears. The blast radius of a problem elsewhere stops at your door. In other words, we follow an anti-Vegas principle—what happens outside your cluster will stay outside.

But true security is layered. Once we've isolated the environment, we defend it from the inside out. We start by asking the hard questions:

Who are you? That's strong authentication, from SCRAM to AWS IAM.

What can you do? That's fine-grained RBAC, enforcing the principle of least privilege.

What if someone gets in? That's encryption everywhere—in transit, at rest, and even in use with Client-Side Field Level Encryption.

How do we lock down the roads? That's network controls like IP access lists and private endpoints.

And how do we prove it? That's granular auditing for a clear, immutable trail.

Every one of these layers reflects defense in depth.

Figure 1. Queryable Encryption.

The history of database security is full of trade-offs between safety and functionality. For decades, the trade-off has been brutal: to run a query, you had to decrypt your data on the server, exposing it to risk. Queryable Encryption—an industry-first searchable encryption scheme developed by MongoDB Research—breaks this paradigm. It allows your application to run expressive queries, including equality and range checks on data that remains fully encrypted on the server. The decryption keys never leave your client. The server maintains encrypted indexes for the fields you wish to query on, and queries can be done entirely on the encrypted data, maintaining the strongest privacy and security of your sensitive data.

By carrying these defenses in the platform itself, security stops being another burden developers have to design around. They get the privacy guarantees, the audit trails, and the compliance, without sacrificing functionality or velocity.

Proof before shipping: Testing tells you that your code works in the cases you've thought to test.

MongoDB Blog

Achieving resilience: Architecture, operations, and proof

Systems don't live in a vacuum. They live in messy realities: network partitions, power outages, kernel panics, cloud control plane hiccups, operator mistakes. The measure of

}

{

Building a Movie Recommendation Engine with Hugging Face and Voyage AI

MongoDB Blog

This guest blog post is from Arek Borucki, Machine Learning Platform & Data Engineer for Hugging Face - a collaboration platform for the machine learning community. The Hugging Face Hub works as a central place where anyone can share, explore, discover, and experiment with open-source ML. HF empowers the next generation of machine learning engineers, scientists, and end users to learn, collaborate and share their work to build an open and ethical AI future together. With the fast-growing community, some of the most used open-source ML libraries and tools, and a talented science team exploring the edge of tech, Hugging Face is at the heart of the AI revolution.

Traditional movie search relies on filtering by genre, actor, or title. But what if you could search by how you feel? Imagine typing:

“something uplifting after a rough day at work”

“a movie that will make me cry”

“I need adrenaline, can’t sleep anyway”

“something to watch with grandma who hates violence”

This is mood-based semantic search: matching your emotional state to movie plot descriptions using AI embeddings.

In this tutorial, you will build a mood-based movie recommendation engine using three powerful technologies: voyage-4-nano (a state-of-the-art open-source embedding model), Hugging Face (for model and dataset hosting), and MongoDB Atlas Vector Search (for storing and querying embeddings at scale).

Why mood-based search? Genre tags are coarse. A “drama” can be heartwarming or devastating. A “comedy” can be light escapism or dark satire. Traditional filters cannot capture these nuances.

Semantic search solves this by understanding meaning. When you search for “feel-good movie for a rainy Sunday”, the system doesn’t look for those exact words. It understands the intent and matches it against plot descriptions that evoke similar feelings.

Architecture overview The system combines three components from the Hugging Face ecosystem with MongoDB:

voyage-4-nano (Hugging Face Hub): Converts text to embeddings (up to 2048 dimensions, we use 1024)

MongoDB/embedded_movies(Hugging Face Datasets): 1500+ movies with plot summaries, genres, cast

MongoDB Atlas Vector Search: Stores embeddings and performs similarity search

}

Understanding voyage-4-nano voyage-4-nano is the smallest model in Voyage AI’s latest embedding series, released with open-weights under the Apache 2.0 license. Voyage AI was acquired by MongoDB, and the Voyage 4 series models are now available through MongoDB Atlas. All models in the series (voyage-4-large, voyage-4, voyage-4-lite, and voyage-4-nano) produce compatible embeddings in a shared embedding space, allowing you to mix and match models within a single use case.

Although voyage-4-nano natively supports embeddings up to 2048 dimensions, we deliberately truncate them to 1024 dimensions using its Matryoshka embedding property. In practice, this provides a strong balance between semantic quality, storage efficiency, and vector search latency, while preserving stable ranking behavior.

Sentence Transformers This tutorial uses Sentence Transformers, a Python library built on top of Hugging Face Transformers. It is specifically designed for working with embedding models and provides a simple API for generating text embeddings.

Why Sentence Transformers instead of raw Transformers? When working with embedding models, you need to handle tokenization, pooling, normalization, and prompt formatting. Sentence Transformers does all of this automatically in a single method call. The code is cleaner, there are fewer potential errors, and you get built-in features like batch processing with progress bars.

Under the hood, Sentence Transformers still uses Hugging Face Transformers to load and run the model.

Configure the development environment Let’s get started!

Create the Project Structure Code Snippet 1 mkdir mood-movie-search 2 cd mood-movie-search 3 mkdir src 4 touch requirements.txt .env

Install dependencies Create the requirements.txt file:

Code Snippet 1 cat =0.109.0 3 uvicorn>=0.27.0 4 pymongo>=4.6.1 5 sentence-transformers>=3.0.0 6 python-dotenv>=1.0.0 7 datasets>=2.16.0 8 torch 9 EOF

Create a Python virtual environment and install dependencies:

to be continued...

{

Load Testing API's on Redshift & Snowflake—A Quick POC

Sameeksha Bhatia · Helpshift Engineering

Load Testing API's on Redshift & Snowflake — A Quick POC

At Helpshift, our data platform follows a Lakehouse architecture, combining the best of both data lakes and data warehouses. This architecture allows us to store and analyze large amounts of raw data in a structured and organized manner, while also providing the scalability and low-cost storage of a data lake. It consists of three key components:

Amazon S3 for storing historical data efficiently.

Amazon EMR for running Spark pipelines on ephemeral compute.

Amazon Redshift as the data warehouse powering customer-facing analytics, which includes Embedded Dashboard analytics, Power BI template apps, and Analytics APIs.

We serve the analytics data models to our customers using AWS Redshift through the following customer channels:

Embedded Analytics dashboard with Luzmo as the BI vendor.

PowerBI integration using REST API as a datasource. PowerBI apps allow customers to also create customized reports on the data models.

REST APIs to query/download analytics data models. These APIs allow custom aggregations and filters. (e.g., hourly or daily rollups, ad hoc filtering, etc.)

We have isolated our read path traffic for the above customer-facing analytics channels in a separate Redshift cluster from the write path traffic. However, in recent months, the team has noticed a few issues during peak hours of read traffic. One of the main challenges that the team faces is performance issues with serving analytics data on the above-mentioned customer channels.

With increasing adoption of our analytics interfaces — Power BI integrations and REST APIs — the Redshift cluster is under growing pressure from concurrent query workloads. These workloads are latency-sensitive and often power downstream operational or reporting workflows. Failures, timeouts, or inconsistent response times can disrupt customer pipelines, delay business-critical decisions, and increase retry or error-handling complexity on the client side. Ensuring predictable performance and horizontal scalability under load is critical to meeting SLAs, minimizing operational noise, and maintaining a consistent analytics experience across all customer-facing endpoints.

Difficulty in Handling High-Concurrent Traffic Redshift offers Multi-Cluster Concurrency Scaling, but it comes at an additional cost and only partially resolves traffic spikes. During peak loads, we observed:

API failures and timeouts due to query queuing and resource contention.

Slow query execution despite concurrency scaling, leading to degraded user experience. Query duration varied from 10 seconds to 3 minutes for similar OLAP queries.

2. Vacuum & Sorting Overhead Under High Load

During high-concurrency execution, we noticed:

Table fragmentation and sort order degradation : As tables are continuously updated with new data, especially under high write throughput, rows become increasingly unsorted. This leads to fragmentation, which degrades scan performance and increases query latencies — particularly for queries relying on sort keys for efficient filtering. High write throughput leads to fragmentation, increasing query execution times.

Increased query execution time due to on-the-fly sorting : Instead of reading pre-sorted blocks, Redshift spent a substantial portion of query execution time re-sorting fragmented data at runtime, resulting in higher and inconsistent query durations.

VACUUM operations failing under load : Redshift requires periodic VACUUM operations to reclaim disk space and restore sort order. However, under heavy read and write workloads, VACUUM processes are deprioritized or delayed in favor of query execution. Since VACUUM is resource-intensive, it competes for the same I/O and CPU resources as live queries. On a saturated cluster, this leads to VACUUM jobs stalling, timing out, or failing entirely — leaving tables fragmented and causing query performance to degrade over time.

These challenges prompted us to explore alternative solutions for handling high-concurrency workloads efficiently. To evaluate alternatives, we conducted a performance benchmarking POC between Redshift and Snowflake using load testing tools like Newman and K6.

Newman and Graphana K6

To effectively simulate traffic and test the performance of our queries, we used Newman and k6, the powerful duo that helped us simulate the load.

Newman is a command-line tool built by Postman to run collections of API requests directly from the terminal. It's particularly useful for automating and running tests for APIs, making it a great tool for testing query performance. It helps to execute postman collections via command line and supports parameterization, i.e., you can specify variables like the number of runs, parallel executions, and delay between requests.

Graphana K6 is an open source tool that is optimized for minimal resource consumption and designed for running high-load performance tests.

}

{

The Original Attention Crisis

Study Hacks · Study Hacks (Cal Newport)

I recently heard from a historian of science at All Souls College, Oxford. He forwarded me an essay he wrote about Nicolaus Steno, a seventeenth-century anatomist and geologist who was later ordained as a Catholic Bishop.

Steno's training as a scholar unfolded in a period challenged by a novel problem: information overload. Here's how the essay describes it:

"Books were a leading distraction in the early modern period—and how envious we should be of those times. From the 1500s onward, with the development of the printing press and the humanist revival of ancient philosophies, knowledge became available at a much greater pace than ever before."

This created pressing questions for aspiring thinkers, including: "How do we decide what to read? How long should we read it for? Must every single chapter be excerpted?"

Part of the solution was the development of "new note-taking techniques," including the copying of excerpts into a master notebook called a book of commonplaces. (For more on this technique, I recommend William Powell's delightful 2010 techno-history, *Hamlet's Blackberry*).

But as the essay on Steno elaborates, better notes weren't enough on their own, as there were simply too many good books available. In response to this reality, Steno, during his university studies in the 1650s, innovated some more advanced attention management strategies:

"[H]e learned to focus on specific themes, rather than letting his mind read multiple things quickly. A 'harmful hastening should be avoided' as he put it. His solution was to 'stick to one topic.'"

In practice, that meant blocking specific moments of time to go through the hardest tasks. As he wrote in his personal notebook, 'before noon nothing must be done except medical things.' ... As Steno told a friend, he took 'almost all the morning hours' to read the works of the Church Fathers and old biblical manuscripts available at the Medici library."

In other words, Steno created a method that combines what we might now call slow productivity , deep work , and time blocking .

The lessons here are clear. The use of our brains to think deeply about meaningful ideas isn't new. It's been at the core of the human experience since the early modern period, when access to sophisticated information first became somewhat widespread.

The best practices developed back then remain the best practices today: avoid overload, focus on one thing at a time, and block off specific hours in your day for your most mentally demanding efforts.

Two weeks ago, a small financial services firm, Citrini Research, published an essay describing a bleak scenario in which AI agents destroy the white-collar job market in the near future. The piece went viral and was cited as a factor in a modest decline of the S&P 500 the next day.

The Citrini essay wasn't the first to float this scenario. In recent weeks, there have been multiple credulous articles and op-eds in major publications proposing similar outcomes (e.g., 1 , 2 , and 3). But the negative impact on the stock market seems to have been the last straw for serious economists who began to push back on these technological ghost stories last week. (I particularly enjoyed a Deutsche Bank analyst who, perhaps borrowing some of my terminology, told the Times that the Citrini article had a "vibes-to-substance ratio" that was "undeniably high.")

If you're looking to reduce your blood pressure about this idea that AI is about to unravel the economy, I suggest reading a detailed response article published by an analyst from the Global Macro Strategies group at Citadel. It begins with a bit of finance geek sarcasm:

"Despite the macroeconomic community struggling to forecast 2-month-forward payroll growth with any reliable accuracy, the forward path of labor destruction can apparently be inferred with significant certainty from a hypothetical scenario posted on Substack..."

It then continues to systematically destabilize the economic naivety of these breathless op-eds and viral essays about how AI will dismantle the economy all at once. It certainly made me feel better.

(If you're looking for additional soothing of your AI anxiety, then you should also check the first episode of my new AI Reality Check podcast series, which I published last Thursday. I have a new episode of the series coming out this upcoming Thursday as well.)

The post *The Original Attention Crisis* appeared first on *Cal Newport* .

}

{

Enhance Your In-IDE Data Browsing Experience With MongoDB

MongoDB Blog

MongoDB is excited to announce the general availability of our enhanced data browsing experience in the MongoDB for Visual Studio (VS) Code extension. This new experience offers a unified workspace for developers to visually browse, query, and edit their data natively, streamlining workflows so they can manage their database right where they write their code.

Evolving the developer workflow The modern developer's workflow is incredibly fast-paced. With developers juggling an average of 14 different tools daily, the cognitive load of constantly jumping between applications can easily disrupt focus. When your application needs to evolve, working with your data shouldn't force a break in your flow state.

As the MongoDB for VS Code extension has grown to nearly 3 million downloads, we've seen firsthand how developers are pushing the boundaries of what an in-IDE (integrated development environment) database tool can do. While developers love accessing their data directly in the editor, we wanted to transform this experience to be even more visual, actionable, and seamless. Instead of switching to external terminals for quick tasks or taking the time to translate familiar MongoDB Shell commands into Extended JSON (EJSON), we are bringing a full-fledged, intuitive data management suite right to your VS Code sidebar.

Exploring what's new in the MongoDB for VS Code extension Here are the key improvements that transform the extension into a complete workflow solution:

Paginated tree view and prescriptive titles Understanding complex data models at a glance is crucial for rapid development. We are transforming the document browsing experience by automatically detecting human-readable fields (like names or emails) to create prescriptive document titles, rather than just displaying standard `_id` hashes. Furthermore, you can now use a structured, paginated tree view to instantly browse collection data from the "Documents" tab, as well as interactively explore playground results when you run a script. This means you get the full context of your collections visually and instantly.

Figure 1. Paginated tree view and prescriptive titles

Powerful action menus and header controls Navigating your data should be inherently actionable. To give you full management capabilities without the need for you to write manual queries, we've added a new action header directly inside the tree view. This header equips you with buttons to instantly insert documents, refresh (to rerun the current query or playground script), sort ascending/descending by `_id`, paginate through results, and even bulk delete to empty a collection.

Additionally, managing individual records is easier than ever. Simply hover over any document within the tree view

to reveal a contextual action menu that allows you to instantly delete, copy, clone, and edit the document natively.

Figure 2. Native action menus

Native editing and shell syntax default We wanted to make interacting with your database as natural as possible.

To remove the friction of translating your commands, we've added a setting that defaults to standard Shell syntax over EJSON for all insert, clone, edit, and clipboard functionalities. This guarantees that any document you copy or any quick fix you make in the extension is instantly compatible with your application code.

Figure 3. Clone action.

Stop context switching and start building Your database tools should adapt to your workflow, not disrupt it. By bringing native data editing, intelligent tree views, and standard Shell syntax directly into your sidebar, we're bridging the gap between writing code and managing data. You no longer have to sacrifice your flow state just to make a quick database fix, verify a playground result, or translate verbose EJSON formats. This overhaul is another step in our commitment to making this MongoDB extension your ultimate command center—empowering you to spend less time wrestling with external tools and more time actually building your application.

}

{

Innovating with MongoDB | Customer Successes, October 2025

MongoDB Blog

It's officially fall! The start of every new season is a perfect time to consider change and new beginnings. While fall might make you think about pumpkin spice and newly chilly evenings, I'm thinking about the latest round of transformations that MongoDB's customers are embracing to thrive in an AI-powered world.

In all seriousness, legacy systems and technical debt are huge challenges: the cost of tech debt has been estimated at almost \$4 trillion dollars. That's trillion with a T! Legacy systems can slow down innovation, create bottlenecks, and make it tough to deliver the seamless, real-time experiences customers increasingly expect. But companies are finding that modernizing their applications isn't just about fixing what's broken—modernization enables them to move faster and innovate for end-users.

That's why I'm incredibly excited to share the recent launch of MongoDB's Application Modernization Platform (AMP). This AI-powered program is designed to help enterprises move beyond outdated infrastructures to embrace a flexible, data-driven future. AMP is a comprehensive approach to modernization that combines smart AI tooling with proven methodologies, enabling businesses to transform their applications from the ground up, moving from legacy monoliths to a more flexible, microservices-based architecture.

In this roundup, we're spotlighting customers who understand the strategic importance of modernization. You'll see how Wells Fargo is using MongoDB to power a new credit card platform, how CSX is ensuring business continuity during a critical migration, how Intellect Design is modernizing its wealth management platform, and how Deutsche Telekom is transforming its B2C digital channels. With MongoDB, customers are showing how integral a modern database is to powering the next generation of applications—and succeeding in the AI era.

Wells Fargo sought to modernize its mainframe-dependent credit card platform to provide a faster, more seamless customer experience and handle an exponential increase in transaction data. The company's legacy system was costly to manage and lacked the scalability needed for its "Cards 2.0" initiative.

To solve this, Wells Fargo built an operational data store (ODS) using MongoDB. This new platform allowed them to adopt reusable APIs, streamline integrations, and move from a monolithic architecture to flexible microservices. The ODS now serves 40% of traffic from external vendors, handling more than 7 million transactions with sub-second service.

By leveraging MongoDB, Wells Fargo was able to jumpstart its mainframe modernization and create curated data products to serve real-time, personalized financial services.

CSX

CSX, a major U.S. railroad company, sought to modernize its critical operations platform, RTOP, by migrating it to the cloud. The challenge was to maintain the platform's 24/7 availability with minimal disruption to its mission-critical, near real-time operations during the transition.

To solve this, CSX selected MongoDB Atlas on Azure and partnered with MongoDB Professional Services. Leveraging the Cluster-to-Cluster Sync (mongosync) feature, the team was able to facilitate continuous data synchronization and complete the entire migration in just a few hours.

The move to MongoDB Atlas has equipped CSX with a more scalable and resilient platform. This modernization effort established a blueprint for migrating other critical applications and helped CSX continue its digital transformation journey toward becoming America's best-run railroad.

Intellect Design, a global fintech company, sought to modernize its wealth management platform to overcome legacy system bottlenecks and multihour batch processing delays. The company's rigid relational database architecture limited its ability to scale and innovate.

To solve this, the company partnered with MongoDB, using our AMP methodology and generative AI tools. This transformation reengineered the platform's core components, resulting in an 85% reduction in onboarding workflow times, allowing clients to access critical portfolio insights faster than ever.

This initiative is the first step in Intellect Design's long-term vision to integrate its entire application suite into a unified, AI-driven service. By leveraging MongoDB Atlas's flexible schema and powerful native tools, the company is now better positioned to deliver smarter analytics and advanced AI capabilities to its customers.

Watch Intellect AI's MongoDB.local Bengaluru keynote presentation to learn how AMP helped them transform outdated systems into scalable, modern solutions.

Deutsche Telekom, a leading telecommunications company, sought to modernize its B2C digital channels, which were fragmented by outdated legacy systems. The company needed to create a unified digital experience for its 30 million customers while improving developer productivity.

By leveraging MongoDB Atlas as part of its Internal Developer Platform, Deutsche Telekom built a robust data infrastructure to unify customer data and power its new digital services. This approach allowed the company to retire legacy systems and reduce its reliance on physical shops and call centers.

The transition to MongoDB Atlas led to a massive surge in digital engagement, with daily customer interactions rising from under 50,000 to approximately 1.5 million. The company's customer data platform now handles up to 15

}

{

Rain: A key-value store for Strava's scale

Derick Yang · Strava Engineering

Much of our heatmaps are built on batch data outputs stored in Rain

At Strava, we love maps — some of our most loved features are nestled on map surfaces. My team, the Geo team, is focused on building and improving these products. On the Geo and Metro teams, we tend to work with large datasets: aggregations of open source map data via OpenStreetMaps, GPS data points from uploaded activities, third-party datasets for properties like elevation, and beyond. This aggregated dataset eventually turns into Geo features we know and love, like the global heatmap, Strava Metro, the routing product, route suggestions, elevation profiles, and points of interest. We perform these data aggregations in a rather hefty data pipeline, run on a regular cadence to ensure we're serving up-to-date geo data.

One of the Geo team's key challenges is efficiently serving large, immutable (write-once, read-many) datasets produced by our pipeline. This is particularly hard for compute-intensive use cases like routing, where:

Write-Optimized vs. Read-Optimized Conflict : Traditional read-optimized data stores struggle with large batch writes without impacting read performance or introducing significant operational complexity.

Cost Constraints : Storing rarely accessed datasets in production databases can be prohibitively expensive — especially for projects like Strava Metro, which are accessed sporadically.

Schema Complexity : Defining schemas externally from the service that uses them can be costly and inflexible for developers.

Our previous solution to large writes used a combination of datastores: PalDB and Cassandra.

PalDB is a binary data format ideal for small datasets. The README states that it is optimized for “side data”, relatively small datasets you read “in passing” on your service. PalDB is unideal, however, for larger datasets. In our case, since each service deployment required downloading the key-value file from S3, our deployments were taking upwards of twenty minutes. Slow deployment hindered rapid iteration for developers. Additionally, the collection of PalDB files occupies a hefty amount of memory on every service instance. For an important service like Routemaster at Strava, this means we end up duplicating the data in memory across potentially dozens of server instances.

Cassandra is advertised as a write-heavy datastore and it can be an effective solution for batch data outputs. Though Strava does still use it as storage for some batch datastores, we can run into problems when we're replacing the full dataset on a regular cadence. In our experience, Cassandra writes from Spark require babysitting to ensure we don't

hit throttling/network/connection limits. Since our builds are full replacements incorporating worldwide changes to the OpenStreetMaps source, this can cause high load to a production datastore.

We embarked on a journey to improve our immutable batch data updates by creating a new service that acts as a key-value store for any dataset generated in Spark. We call this service Rain, because it distributes data from the cloud onto client services.

At its core, Rain behaves analogously to the cache in your operating system, except on a distributed systems level. Just as the OS loads file blocks into cache, Rain loads subsets of your Spark-output dataset on S3 (“the filesystem”) into Redis (“the L1/L2 cache”). Client services retrieve the datasets via a rich client that calls a service. In code, Rain has three primary components: a Spark writer API, a library for reads, and a Thrift service, shown below:

What does this structure enable? First, it allows us to tap into the power of Spark distributed write, which is optimized for Parquet outputs. Second, it allows us to perform immutable data hot swap by simply changing the reference path for the dataset, which is provided through an admin interface. Third, it helps us save on data costs by using a LRU (least recently used) Redis cache and using a single distributed datastore rather than a duplicated in-memory store on each service instance.

Writing a Rain table

Strava uses Apache Spark to run our data pipelines and write Rain tables. For datasets we update regularly, we typically schedule refreshes with Apache Airflow. We output the data of a fully refreshed table to a clean prefix in S3 and update a file representing a pointer to the table data to point at this new prefix.

Rain is responsible for regularly checking if this reference data pointer has been updated to point to new table data. At the Rain service level, we use a Caffeine cache to expire the pointer pointing to the old dataset. This enables us to replace the immutable dataset in the background, without redeploying any component of the system. The table pointer update is simply an API call to the Rain server. The server then updates the pointer. When Caffeine expires the pointer at the old dataset, Rain will start reading data from the new dataset.

Key/value pairs in a Rain table can consist of any arbitrary Kryo-serializable case classes. This means that services can be responsible for defining their own schemas, without involving a database administrator. We write, store, and transfer data using a byte array format, and clients define the key/value types and ensure they are Kryo serializable. The Spark client and client server reader handle object serialization and deserialization, respectively.

The client library is responsible for deserializing the byte array results that Rain returns into types that the client can

}

standard-article(title: [Mapping microwave relay links from video], author: [Oona Räisänen], source-name: [Oona Raisanen (windytan)], images: (), paragraphs: ([Radio networks are often at least partially based on microwave relay links . They're those little mushroom-like appendices growing out of cell towers and building-mounted base stations. Technically, they're carefully directed dish antennas linking such towers together over a line-of-sight connection. I'm collecting a little map of nearby link stations, trying to find out how they're interconnected and which network they belong to.], [Circling around], [We can find a rough direction for any link antenna by approximating a tangent for the dish shroud surface from position-stamped video footage taken while circling the tower. Optimally we would have a drone make a full circle around the tower at a constant distance and elevation to map all antennas at once; but if our DJI Phantom has run out of battery, a GPS positioned still camera at ground level will also do.], [The rest can be done manually, or using Hough transform and centroid calculation from OpenCV. In these pictures, the ratio of the diameters of the concentric circles is a sinusoid function of the angle between the antenna direction and the camera direction. At its maximum, we're looking straight at the beam. (The ratio won't max out at unity in this case, because we're looking at the antenna slightly from below.) We can select the frame with the maximum ratio from high-speed footage, or we can interpolate a smooth sinusoid to get an even better value.], [This particular antenna is pointing west-northwest with an azimuth of 290°], [What about distance?], [Because of the line-of-sight requirement, we also know the maximum possible distance to the linked tower, using the formula $7140 \times \sqrt{4 / 3 \times h}$ where h is the height of the antenna from ground. If the beam happens to hit a previously mapped tower closer than this distance, we can assume they're connected!], [This antenna is communicating to a tower not further away than 48 km. Judging from the building it's standing on, it belongs to a government trunked radio network.],), insert-map: (:), word-count: 338, edited-for-length: false, debug-mode: false,)

{

standard-article(title: [Polygraphs have major flaws. Are there better options?], author: [Sarah Scoles, Undark Magazine], source-name: [Ars Technica OpenForum], images: (), paragraphs: ([When George W. Maschke applied to work for the FBI in 1994, he had already held a security clearance for over 11 years. The government had deemed him trustworthy through his career in the Army. But soon, a machine and a man would not come to the same conclusion.], [His application to be a special agent had passed initial muster. And so, in the spring of 1995, according to his account, he found himself sitting across from an FBI polygraph examiner, answering questions about his life and loyalties.], [He told the truth, he said in an interview with Undark. But in a blog post on his website, he recalled the examiner told him that the polygraph machine—which measured some of Maschke's physiological responses—indicated that he was being deceptive about keeping classified information secret, and about his contacts with foreign intelligence agencies.], [Read full article],), insert-map: (:), word-count: 145, edited-for-length: false, debug-mode: false,)

Safely Modifying Your Hosts File with Gas Mask

Curalate Engineering

Sometimes the DNS for a specific domain on your machine needs to point somewhere else – at Curalate, we test changes microservices locally before shipping them, which could require redirecting requests to look at that local instance. One way to do this is by adding an entry like 127.0.0.1 some.service.curalate.com to /etc/hosts .

id="why-use-a-hosts-file-manager">Why use a hosts file manager?

In most cases, it's not advised to directly modify /etc/hosts . Because it's buried deep into the filesystem, it's easy to forget you've modified it, which can lead to numerous problems ranging from annoying to dangerous. Also, danger aside, it can begin to get messy and complex if you have a lot of entries to manage. Think of even just fifteen lines you're constantly commenting/uncommenting to represent the configuration you need at a given moment. This would be insanity. Gas Mask, a simple UI-based hosts file manager, allows you to set up different hosts files, while making it plainly obvious which hosts file is currently activated on your system via the OS Menu bar.

id="installation-instructions">Installation instructions

Go to <https://github.com/2ndalpha/gasmask> and download the latest version.

Unpack and install.

On first-run, the only hosts file listed will be Original File which is the /etc/hosts file you'll no longer be modifying.

id="creating-your-first-host-file">Creating your first host file

Create a new hosts file and name it something that makes sense.

Add the test entry 127.0.0.1 google.com and save. The format of these entries is .

Activate that hosts file. Gasmask substitutes in this file at /etc/hosts .

You may either need to flush your DNS cache or just restart the browser.

To test it out, go to google.com in the browser.

What happens now is, when the browser goes to get the IP for google.com, the OS sees the matching entry in your hosts file, then refers to 127.0.0.1 (your local computer) to make the request – which will fail.

Go ahead and reactivate your Original File, restart the browser, and you should be able to access google.com as expected.

id="use-the-menubar-icon">Use the Menubar icon

It's easy to forget to flip your hosts file back. You'll end up spending 45 minutes on what you think is a bug, that doesn't reproduce for anyone else, only to realize your hosts file is sending requests someplace else.

Next time you reach to modify that hosts file, consider integrating Gas Mask into your development workflow to keep it maintained and safe from unintended state.

}

{

ANALYSIS

Gramophone audio from photograph, revisited

Oona Räisänen · Oona Raisanen (windytan)

“I am the atomic powered robot. Please give my best wishes to everybody!”

Those are the words uttered by Tommy, a childhood toy robot of mine. I’ve taken a look at his miniature vinyl record sound mechanism a few times before (#1 , #2), in an attempt to recover the analog audio signal using only a digital camera. Results were noisy at best. The blog posts resurfaced in a recent IRC discussion which inspired me to try my luck with a slightly improved method.

Source photo

I will be using an old photo of Tommy’s internal miniature record I already had from previous adventures in 2012. I don’t want to perform another invasive operation on Tommy to take a new photograph, as I already broke a plastic tab last time I opened him. But it also means I don’t have control over the photographing environment. It’s part of the challenge.

The picture was taken with a DSLR and it’s an uncompressed 8-bit color photo measuring 3000 by 3000 pixels. There’s a fair amount of focus blur, chromatic aberration and similar distortions. But at this resolution, a clear pattern can be seen when zooming into the grooves.

This pattern superficially resembles a variable-area optical audio track seen in old film prints, and that’s why I previously tried to decode it as such. But it didn’t produce satisfactory results, and there is no physical reason it even should. In fact, I’m not even sure as to which physical parameter the audio is encoded in – does the needle move vertically or horizontally? How would this feature manifest itself in the photograph? Do the bright blobs represent crests in the groove, or just areas that happen to be oriented the right way in this particular lighting?

To make the grooves a little easier to follow I first unwrapped the circular record into a linear image. I did this by remapping the image space from polar to 9000-wide Cartesian coordinates and then resampling it with a windowed sinc kernel:

Mapping the groove path

It’s not easy to automatically follow the groove. As one would imagine, it’s not a mathematically perfect spiral. Sometimes the groove disappears into darkness, or blurs into the adjacent track. But it wasn’t overly tedious to draw a guiding path manually. Most of the work was just copy-pasting from a previous groove and making small adjustments.

I opened the unwrapped image in Inkscape and drew a colored polyline over all obvious grooves. I tried to make sure a polyline at the left image border would neatly continue where the previous one ended on the right side.

The grooves were alternatively labeled as ‘a’ and ‘b’, since I knew this record had two different sound effects on interleaved tracks.

This polyline was then exported from Inkscape and loaded by a script that extracted a 3-7 pixel high column from the unwrapped original, centered around the groove, for further processing.

Pixels to audio

I had noticed another information-carrying feature besides just the transverse area of the groove: its displacement from center. The white blobs sometimes appear below or above the imaginary center line.

I had my script calculate the brightness mass center (weighted y average) relative to the track polyline at all x positions along the groove. This position was then directly used as a PCM sample value, and the whole groove was written to a WAV file. A noise reduction algorithm was also applied, based on sample noise from the silent end of the groove.

The results are much better than what I previously obtained (see video below, or mp3 [here](#)):

Future ideas

Several factors limit the fidelity and dynamic range obtained by this method. For one, the relationship between the white blobs and needle movement is not known. The results could possibly still benefit from more pixel resolution and color bit depth. The blob central displacement (insofar as it is the most useful feature) could also be more accurately obtained using a Gaussian fit or similar algorithm.

The groove guide could be drawn more carefully, as some track slips can be heard in the recovered audio.

Opening up the robot for another photograph would be risky, since I already broke a plastic tab before. But other ways to optically capture the signal would be using a USB microscope or a flatbed scanner. These methods would still be only slightly more complicated than just using a microphone! The linear light source of the scanner would possibly cause problems with the circular groove. I would imagine the problem of the disappearing grooves would still be there, unless some sort of carefully controlled lighting was used.

}

{

Avoiding Digital Productivity Traps

Study Hacks · Study Hacks (Cal Newport)

Last week in this newsletter, I summarized some interesting results from a study that analyzed the behavior of 164,000 knowledge workers. It found that introducing AI tools increased administrative tasks by more than 90% while reducing deep work effort by almost 10%.

The problem, I concluded, was that digital productivity tools sometimes speed up the wrong tasks, which might feel efficient in the moment, but lead us to accomplish less over time. As I emphasized, AI is not the only technology to produce this paradoxical side effect—we saw something similar with email, mobile computing, and online meeting software as well.

So, what's the solution to avoid these traps?

In today's episode of my podcast, I suggested three ideas that might help. I want to summarize them here as well:

Idea #1: Use a Better Scoreboard

Make sure you measure what actually matters in your job. If you're a professor at a research institution, for example, this might be the number of papers you publish per year. If you're a team manager, it might be the number of priority projects completed per month.

When you introduce new digital productivity tools into your workflow, don't focus too much on their impact on individual tasks (e.g., "Wow! That email was much faster to send than a fax," or "AI just finished a task in 20 minutes that would have taken me 3 hours!"). Pay attention instead to your scoreboard. If you're not producing more valuable output than before, the tool isn't really making you more productive.

Idea #2: Focus on the Right Bottlenecks

If you look closer at many knowledge work projects, you'll identify a key bottleneck that determines how fast they can be accomplished. If you want to become more productive, you should look for ways to deploy tools that improve this specific step.

When working on Deep Work, for example, I spoke with a prominent Wharton professor who told me that one of the keys to publishing journal papers in his field was access to interesting data sets. He published more papers per year than most of his peers, largely because he spent more time building relationships with companies and institutions in search of good data. This was the bottleneck for his work.

Accordingly, any tool that could help him cultivate more such relationships and gather better data from the relationships he had already formed would directly improve his productivity. Compare this, for example, to using Claude Code to speed up the process of producing plots for his papers. This might, in limited windows of time, make his job

more convenient, but not necessarily increase the number of papers he publishes per year.

Idea #3: Separate Deep from Shallow Work

My final idea is the simplest: on your daily calendar, clearly separate time for focused effort that directly produces value from administrative, logistical, and collaborative tasks. In this way, if a digital productivity tool ends up accidentally increasing the volume of shallow work you face each day, you'll limit the damage to your ability to make progress on important projects.

This makes it easier to experiment with different tools without worrying that you might end up — like many of the subjects in the study cited above — suddenly overwhelmed by the ultra-fast processing of minutiae while the big things slowly languish.

The post Avoiding Digital Productivity Traps appeared first on Cal Newport .

}

{

Vision RAG: Enabling Search on Any Documents

MongoDB Blog

Information comes in many shapes and forms. While retrieval-augmented generation (RAG) primarily focuses on plain text, it overlooks vast amounts of data along the way. Most enterprise knowledge resides in complex documents, slides, graphics, and other multimodal sources. Yet, extracting useful information from these formats using optical character recognition (OCR) or other parsing techniques is often low-fidelity, brittle, and expensive.

Vision RAG makes complex documents—including their figures and tables—searchable by using multimodal embeddings, eliminating the need for complex and costly text extraction. This guide explores how Voyage AI's latest model powers this capability and provides a step-by-step implementation walkthrough.

Vision RAG: Building upon text RAG Vision RAG is an evolution of traditional RAG built on the same two components: retrieval and generation.

In traditional RAG, unstructured text data is indexed for semantic search. At query time, the system retrieves relevant documents or chunks and appends them to the user's prompt so the large language model (LLM) can produce more grounded, context-aware answers.

Figure 1. Text RAG with Voyage AI and MongoDB.

Text RAG with Voyage AI and MongoDB Enterprise data, however, is rarely just clean plain text. Critical information often lives in PDFs, slides, diagrams, dashboards, and other visual formats. Today, this is typically handled by parsing tools and OCR services. Those approaches create several problems:

Significant engineering effort to handle many file types, layouts, and edge cases

Accuracy issues across different OCR or parsing setups

High costs when scaled across large document collections

Next-generation multimodal embedding models provide a simpler and more cost-effective alternative. They can ingest not only text but also images or screenshots of complex document layouts, and generate vector representations that capture the meaning and structure of that content.

Vision RAG uses these multimodal embeddings to index entire documents, slides, and images directly, even when they contain interleaved text and images. This enables them to be searchable via vector search without requiring heavy parsing or OCR. At query time, the system retrieves the most relevant visual assets and feeds them, along with the text prompt, into a vision-capable LLM to inform its answer.

Figure 2. Vision RAG with Voyage AI and MongoDB.

Vision RAG with Voyage AI and MongoDB As a result, vision RAG enables LLM-based systems with native access to rich, multimodal enterprise data, while reducing engineering complexity and avoiding the performance and cost pitfalls associated with traditional text-focused preprocessing pipelines.

Voyage AI's latest multimodal embedding model The multimodal embedding model is where the magic happens. Historically, building such a system was challenging due to the modality gap. Early multimodal embedding models, such as contrastive language-image pretraining (CLIP)-based models, processed text and images using separate encoders. Because the outputs were generated independently, results were often biased toward one modality, making retrieval across mixed content unreliable. These models also struggled to handle interleaved text and images, a critical limitation for vision RAG in real-world environments.

Voyage-multimodal-3 adopts an architecture similar to modern vision-capable LLMs. It uses a single encoder for both text and visual inputs, closing the modality gap and producing unified representations. This ensures that textual and visual features are treated consistently and accurately within the same vector space.

Figure 3. CLIP-based architecture vs. voyage-multimodal-3's architecture.

CLIP-based architecture vs. voyage-multimodal-3's architecture This architectural shift enables true multimodal retrieval, making vision RAG a viable and efficient solution. For more details, refer to the voyage-multimodal-3 blog announcement.

Implementation of vision RAG Let's take a simple example and showcase how to implement vision RAG. Traditional text-based RAG often struggles with complex documents, such as slide decks, financial reports, or technical papers, where critical information is often locked inside charts, diagrams, and figures.

By using Voyage AI's multimodal embedding models alongside Anthropic's vision-capable LLMs, we can bridge this gap. We will treat images (or screenshots of document pages) as first-class citizens, retrieving them directly based on their visual and semantic content and passing them to a vision-capable LLM for reasoning.

To demonstrate this, we will build a pipeline that extracts insights from the charts and figures of the GitHub Octoverse 2025 survey, which simulates the type of information typically found in enterprise data.

The Jupyter Notebook for this tutorial is available on GitHub in our GenAI Showcase repository. To follow along, run the notebook in Google Colab (or similar), and refer to this tutorial for explanations of key code blocks.

Step 1: Install necessary libraries First, we need to set up our Python environment. We will install the voyageai client

}

{

Celebrating Excellence: MongoDB Global Partner Awards 2025

MongoDB Blog

In a world being reshaped by AI and rapid technological change, one thing is clear: our partners are powering the future with MongoDB. Together, we help customers modernize legacy systems, solve challenges from security to budget constraints, and build the next wave of AI-powered applications.

That's why we're proud to announce the annual MongoDB Global Partner Awards — celebrating partners who led the way in 2025. From pioneering AI and modernization to advancing public sector innovation to building bold go-to-market collaborations, these partners set the standard for excellence. Their leadership doesn't just move the needle — it redefines what's possible.

This post is also available in: Deutsch , Français , Español , Português , Italiano , 繁體中文 , 简体中文 .

Global Cloud Partner of the Year: Microsoft

We are proud to recognize Microsoft for exceptional year-over-year growth as MongoDB's Global Cloud Partner of the Year. Together, MongoDB and Microsoft have delivered strong momentum across industries such as healthcare, telecommunications, and financial services, helping organizations build great applications that deliver exceptional customer experiences.

Microsoft's deep commitment to collaboration, customer success, and cloud leadership makes it an indispensable part of MongoDB's partner ecosystem. The strength of the partnership continues to grow; in fact, MongoDB was recently selected as a Microsoft partner for a "Unify your data solution play," which enables customers to benefit from the joint integrations and go-to-market (GTM) resources between MongoDB Atlas on Azure and Native Microsoft services.

Global AI Cloud Partner of the Year: Amazon Web Services (AWS)

AWS has been a driving force in helping customers unlock the full potential of AI with MongoDB, highlighted by our work with Novo Nordisk , who leveraged Amazon Bedrock and MongoDB Atlas to build an AI solution that cut one of their most time-intensive workflows from 12 weeks to 10 minutes. The work with Novo Nordisk is just one example of many that showcases the power of our partnership to create business differentiation for customers in the gen AI era.

MongoDB was also a generative AI Competency launch partner for AWS, further tightening our collaboration in AI. From breakthrough generative AI use cases and beyond, our partnership empowers organizations to move faster, innovate more boldly, and transform with confidence. Together, AWS and MongoDB are shaping what's possible in the AI era.

Global Cloud GTM Partner of the Year: Google Cloud

Google Cloud is being honored for accelerating new business through impactful joint GTM initiatives. MongoDB's partnership with Google Cloud has set the standard for meaningful collaboration—driving new business and delivering impact across some of the world's most complex global enterprises. The joint Google Cloud and MongoDB Sales Development Representative program has been the cornerstone of this success, ensuring early-stage talent get the opportunity to work with the largest organisations in the world whilst learning a sales playbook that will serve them well for the rest of their career. Google Cloud continues to be a driving force in MongoDB's global growth thanks to its joint commitment to innovative GTM strategies.

Global Systems Integrator Partner of the Year: Accenture

Accenture has demonstrated exceptional commitment as a Global SI Partner, establishing a dedicated center of excellence for MongoDB within its software engineering service line.

Together, MongoDB and Accenture have delivered transformative customer outcomes across industries, from payment modernization for a leading bank to data transformation for a major manufacturer. Meanwhile, closer collaboration with Accenture's BFSI business unit has continued to fuel global customer success. By combining MongoDB's modern database platform with Accenture's deep industry expertise, our partnership continues to help customers modernize, unlock data-driven insights, and accelerate digital transformation at enterprise scale.

Global Public Sector Partner of the Year: Accenture Federal Services

Accenture Federal Services has played a pivotal role in advancing MongoDB's presence in the public sector. Thanks to its scale, expertise, and focus on customer outcomes, it has driven remarkable year-over-year growth and has supported critical government missions in coordination with MongoDB.

MongoDB and Accenture Federal Services are helping government agencies meet their efficiency goals by modernizing legacy applications, seamlessly consolidating platforms, and streamlining architectures, all while reducing costs. We are excited to have Accenture Federal Services as a key sponsor of our inaugural MongoDB Public Sector Summit in January 2026.

Global Tech Partner of the Year: Confluent

Confluent —the data streaming platform built by the co-creators of Apache Kafka®—continues to be a strategic partner with more than 550 joint customer deployments delivering impact across industries worldwide. Over the past year, MongoDB and Confluent have strengthened global go-to-market (GTM) alignment, focusing acceleration of co-sell engagement across EMEA and APAC.

}

{

Ultrasonic investigations in shopping centres

Oona Räisänen · Oona Raisanen (*windytan*)

I can't remember how I first came across these near-ultrasonic 'beacons' ubiquitous in PA systems. I might have been scrolling through the audio spectrum while waiting for the underground train; or it might have been the screeching 'tinnitus-like' sensation I would often get near the loudspeakers at a local shopping centre.

Whatever the case, I learned that they are called pilot tones. Many multi-loudspeaker PA systems (like the Zenitel VPA and Axys End of Line detection unit) employ these roughly 20-kilohertz tones to continuously measure the system's health status: no pilot tone means no connection to a loudspeaker. It's usually set to a very high frequency, inaudible to humans, to avoid disturbing customers.

However, these tones are powerful and some people will still hear them, especially if the frequency gets below 20 kHz. There is one such system at an uncomfortable 19.595 kHz in my city; it's marked green in the graph above. I've heard of several other people that also hear the sound. I don't believe it to be a sonic weapon like The Mosquito; those use even lower frequencies, down to 17 kHz. It's probably just a misconfiguration that was never fixed because the people working on it couldn't experientially confirm any issue with it.

Hidden modulation.

Pretty quickly it became apparent that this sound is almost never a pure tone. Some kind of modulation can always be seen wiggling around it in the spectrogram. Is it caused by the background music being played through the PA system? Is it carrying some information? Or is it something else altogether?

I've found at least one place where the tone appears to be amplitude modulated by the lowest frequencies in the music or commercials playing. It's probably a side effect of some kind of distortion.

Here's a spectrogram plot of this amplitude modulation around the strong pilot tone. It's colour-coded so that the purple colour is coming from the right microphone channel and green from the left. I'm not quite sure what the other purple horizontal stripes are here.

But this kind of modulation is rare. It's more common to see the tone change in response to things happening around you, like people moving about. More on that in the following.

Doppler-shifted backscatter.

Look what happens to the pilot tone when a train arrives at an underground station:

The wideband screech in the beginning is followed by this interesting tornado-shaped pattern that seems to have a lot

of structure inside of it. It lasts for 15 seconds, until the train comes to a stop.

It's my belief that this is backscatter, also known as reverb, from the pilot tone reflecting off the slowing train and getting Doppler-shifted on the way. The pilot tone works as a continuous-wave bistatic sonar. Here, the left microphone (green) hears a mostly negative Doppler shift whereas the right channel (purple) hears a positive one, as the train is passing us from right to left. An anti-aliasing filter has darkened the higher frequencies as I wasn't yet aware I would find something interesting up here when recording.

A zoomed-in view of this cone reveals these repeating sweeps from positive to negative and red to green. Are they reflections off of some repeating structure in the passing train? The short horizontal bursts of constant tone could then be surfaces that are angled in a different direction than the rest of the train. Or perhaps this repetition reflects the regular placement of loudspeakers around the station?

Moving the microphone.

Another interesting experiment: I took the lift to another floor and recorded the ride from inside the lift. It wasn't the metal box type, the walls were made of glass, so I thought the pilot tone should be at least somewhat audible inside. Here's what I got during the 10-second ride. It's a little buried in noise.

Skater calculation.

For the next experiment I went into the underground car park of a shopping centre. I stood right under a PA loudspeaker and recorded a skateboarder passing by. A lot of interesting stuff is happening in this stereo spectrogram!

First of all, there seems to be two pilot tones, one at 19,595 Hz and a much quieter one at 19,500 Hz. Are there two different PA systems in the car park?

Second, there's a clear Doppler shift in the reverb. The frequency shift goes from positive to negative at the same moment that the skater passes us, seen as the wideband wheel noise changing color. It looks like the pattern is also 'filled in' with noise under this Doppler curve. What all information can we find out just by looking at this image?

If we ignore the fact that this is actually a bistatic doppler shift we could try and estimate the speed using a formula on Wikipedia. It was pretty chilly in the car park, I would say 15 °C. The speed of sound at 15 °C is 340 m/s. The maximum Doppler shift here seems to be 350 Hz. Plugging all these into the equation we get 11 km/h, which sounds like a realistic speed for a skater.

Why is it filled in? My thought is these are reflections off different points on our test subject. There's variation in the reflection angles and, consequently, magnitudes of the velocity component that causes frequency shifting, down to nearly zero Hz.

}

{

Embedding Teamwork Chat

Dawid Myslak · Teamwork Engineering Room

A couple of weeks ago we introduced a new feature in Teamwork Chat called Project Rooms .

Project Rooms are a great way to create dedicated real-time communication channels for specific projects. Our next priority was to take this feature even further.

Project Rooms in Teamwork Chat.

Currently Project Rooms can be used inside Teamwork Chat, but our goal was to make them accessible directly in Teamwork Projects. To achieve this, we considered the following two options:

Decouple only the important bits and pieces from the existing codebase of Teamwork Chat and deliver them as a standalone module.

Create a brand new solution that follows component based architecture which could potentially be the future core codebase for the next generation of Teamwork Chat.

After looking into this for a while, we decided that the second option will let us deliver a better quality solution. Our main concern with decoupling the existing codebase was the fact that we didn't use a component based architecture in the existing client. Unfortunately at the time of the initial implementation of the app, Knockout.js components weren't a common choice.

New tech stack

Earlier this year at Teamwork.com we made the decision that all our development teams will start using Vue.js as the main front-end framework. We also started working on creating a new internal build system based on Webpack, which allows us to use the latest front-end tools and makes the development process significantly more productive.

Together with Vue.js we started using Vuex, which is a state management library inspired by Flux architecture. Another big change for us was dropping CoffeeScript and replacing it with the next generation of JavaScript. JavaScript ES2015+ comes with great new features and syntax, but also offers incredible editor support and great ESLint that makes our code more consistent across the team.

New technology stack for embedded Teamwork Chat.

Luckily, during the development of an existing Teamwork Chat client we split some parts of the code into Node.js modules and we were able to use them with our new codebase.

Sneak peek

As a result of a few weeks of intensive development, we've been able to implement the core functionality of the Teamwork Chat conversation component.

Initial preview of conversation component.

Taking care of performance

Embedded Teamwork Chat will be an addition for Teamwork Projects. We spent a lot of time making sure that we won't impact its performance.

Starting with a brand new codebase allowed us to improve the implementation of particular parts of Teamwork Chat. We were also able to use latest Web APIs which resulting is a significant improvement in app performance, for example thanks to Intersection Observer API our infinite scroll feature is now much better!

Making it faster and lighter!

Putting all pieces together

Currently, the Teamwork Projects front-end is built with Knockout.js, so we had to find a way to integrate our new code with the existing product.

We decided to make it available as a private Node.js module and we wrapped the entire Vue.js app with the Knockout.js component, so it can be easily included and used inside Teamwork Projects.

Embedded Teamwork Chat preview.

Show me the code!

Including embedded Teamwork Chat is just as easy as that:

As you can see, we simply follow the Knockout.js components syntax and pass the project ID value, that's it! Our module will do all the hard work: it will check if the given project has a dedicated Project Room enabled, and if it does, it will bootstrap our Vue.js app inside — voilà!

Combining Vue.js and Knockout.js together

Here you can find the code snippet (simplified version) of our solution for integrating Vue.js and Knockout.js together:

<https://medium.com/media/e575cbb100c88cff6c1f9be2f114f0a3/href>

We're very excited about Embedded Teamwork Chat and we aim to deliver it by the end of this year!

If you have any questions, leave a comment below.

If you like the sound of working at Teamwork , get in touch — we're hiring !

}

{

Clojure Runs ONNX AI Models Now - Join the AI fun!

Dragan Djuric

Hello, Clojurians! I haven't written here in a long time. Was I tired? Is anybody reading blogs anymore? Who knows. But that was not the main reason.

I've been working on several Clojure projects sponsored by the Clojurists Together Foundation. I did a ton of things, but after all this programming, I was kinda tired, and kept slugging when it comes to telling people about the work done! That's not very smart, but you know how it goes... :) But, then, if we don't tell people about awesome software that we have, nobody is going to use it, so finally I had to stop kicking this down the road, sit, and write the first post. It's been long overdue, so expect more posts soon!

id="orge2dbf4b">ONNX Runtime in one line of Clojure

The most recent thing I'm currently working on started its life as Clojure ML (again, superthanks to Clojurists Together for sponsoring this). I proposed to create a human-friendly Clojure API for AI/DL/ML models, and back it by the first implementation, in this case based on ONNX Runtime . Of course, it should all be integrated into existing Clojure libraries, and follow the Clojure way of doing stuff as much as possible!

The idea is to get an existing, pre-trained ML model previously exported to the ONNX format from whatever technology the authors chose (which in today's world is typically Python and PyTorch), and put it into production in Clojure and JVM. It should be seamless and in-process, without any clunky interoperability, copy, translation, etc. Of course, our Clojure numerical libraries fully support GPU computing, so it goes without saying that we want that, too! Just to be clear, we do not use nor need any Python or Python interop for this, we use the ONNX Runtime 's underlying C library .

Nice idea, but what parts of this well intended story can we evaluate in our REPLs right now? At least some promising demo? Are we on the trail? To access that AI goodness, we surely have to do a sophisticated dance? Are the steps hard to learn? Do we need to watch carefully for slippery floor? Is it accessible to mere mortals?

Here's the gist:

"Wait, what?", you'll say. One function? One tini, tiny, function, with one laughingly trivial argument? Is that an API? What does such trivial API do? "Now you confused me!", you'll scratch your head. It's just a stick.

I hope I've also intrigued you, so please keep reading to see it in action (this post is actually generated from a live REPL session, so the example is fully executable, not just interesting bits on the table, and a ton of complex boilerplate under a Persian rug).

id="org2a73fbe">Hello World, the MNIST image recognition model

For this recipe, you'll need the following ingredients: Deep Diamond tensors (one cup), Deep Diamond network (one slice), one Neanderthal transfer! function for moving data around for demo purposes, and that's it! Oh, yes, don't forget the new onnx function. We load the native namespace, and the right Deep Diamond engine is set up for our system (yes, even on Mac OS, thanks to Clojurists Together !).

```
class="org-src-container">
(require '[uncomplicate.neanderthal.core :refer [transfer!
iamax ] ] '[uncomplicate.diamond [ tensor :refer [ tensor
desc ] ] [ dnn :refer [ network ] ] [ onnxrt :refer [ onnx ] ] ]
'[uncomplicate.diamond.native ] )
```

id="orgc0eecd9">The ONNX model

We evaluate the onnx function, and it loads the model.

```
#'user/mnist-onnx
```

Sure, that's easy, but how is that useful? Well, the result is a function. This function has just been set up with ONNX internals, so now it can create Deep Diamond network layers and fit in with the rest of the Tensor-y stuff that DD already provides.

The ONNX Runtime model revolves around environment, session, input and output tensors, type info, and a lot of other stuff and brittle ceremony. Sure, sometimes you need to reach these internals, and diamond-onnxrt provides clojurized internals API even for that. However, it can sing the main song, and set all the right arguments at the right places for you. Even the onnx function supports option map, where you can tell what you like, and it will take care to configure ONNX to do the right thing, but this is a story for another article.

id="org56b31c2">The rest is the usual Deep Diamond stuff, which is simple as beans!

The MNIST dataset specifies images of hand-written digits, in just one grayscale channel, each (28×28) pixels a challenging task for 1989's USPO and the technology from back then, but a hello world level stuff for today's accelerated libraries (still, keep in mind that if you tried to code even this easy example without such libraries, you'll be surprised how slow that can be!).

We create a tensor descriptor for such input (this step can be left out, but I'm being pedantic to accommodate beginners):

```
#'user/input-desc
```

Next, we create a reusable abstract network blueprint, that can then create concrete networks tailored for training, or optimized for inference, that is classifying MNIST images. Normally, we would have to train these networks, or load the parameters from somewhere, but in this case it contains only of the onnx model, which had already been trained

}

standard-article(title: [Long fingernails vs. touchscreens: This nail polish could help], author: [Jennifer Ouellette], source-name: [Ars Technica OpenForum], images: (), paragraphs: ([The rise of touchscreen technology has been a boon in many respects, but for people with long fingernails, there can be issues with the capacitive variety since fingernails are non-conductive and thus don't register on the screen as a touch. One can use a stylus, of course, or simply use the finger pad under the nail, but ideally it would be nice to be able to use one's fingernail. A conductive nail polish might do the trick, according to research presented at a meeting of the American Chemical Society in Atlanta, Georgia.], [The work began as a special project for Manasi Desai, an undergraduate at Centenary College of Louisiana who has an interest in cosmetic chemistry and decided to investigate ways to make fingernails compatible with touchscreen technology. There are a few existing conductive nail polishes that rely on spiking a clear polish with carbon nanotubes, conductive polymers, or metallic particles. And in 2013 and 2014, a proposed press-on false fingernail with a capacitive tip was showcased at CES in Las Vegas, although the technology doesn't seem to be commercially available.], [Desai reasoned that existing polishes rely on additives that could be dangerous if inhaled, as well as having a limited shade range given that they impart a black or metallic shimmer. Working with her supervisor, organometallic chemist Joshua Lawrence, Desai decided to try to create a clear, colorless nail polish that didn't use any toxic materials and could be applied over any manicure.], [Read full article],), insert-map: (:), word-count: 249, edited-for-length: false, debug-mode: false,)

standard-article(title: [Study says roads bring more fires to forests; USDA wants more roads to fight fires], author: [Zoë Rom, Inside Climate News], source-name: [Ars Technica OpenForum], images: (), paragraphs: ([When the Trump administration announced plans last year to rescind a rule limiting roadbuilding and timber harvests on millions of acres of national forests and grasslands, officials called the repeal necessary to prevent and manage wildfires.], [But as the US Department of Agriculture prepares to release its draft environmental impact statement for the rescission, that justification is unraveling. And many critics of the move see the claim that roads are needed to fight fires in remote forests as cover for a giveaway to the timber industry.], [On average, about 8 million acres have burned each year between 2017 and 2021, according to the Congressional Budget Office, nearly double the average from 1987 to 1991. Wildfires on federal lands average about five times the size of those in the rest of the country, leading some of the nation's top land managers to argue that national forests are a front line for fighting the nation's steep increase in wildland blazes.], [Read full article],), insert-map: (:), word-count: 163, edited-for-length: false, debug-mode: false,)

A conductive nail polish might do the trick, according to research presented at a meeting of the American Chemical Society in Atlanta, Georgia.

Jennifer Ouellette

{

One app was tanking my PC performance, and I had no idea until I checked this

Kanika Gogia · MakeUseOf

I knew something was not right with my PC. No, it wasn't unstable. It didn't even crash randomly. However, everything felt delayed. My apps would take longer to open, my browser stuttered at times, and even simple tasks like typing lagged behind my thoughts. When this started happening frequently, I blamed my hardware. I thought maybe it was actually time to upgrade RAM, or even my entire machine. In reality, it turned out that hardware wasn't the problem. There was one app silently draining performance in the background. And I had no idea until I decided to open a simple Windows tool on my PC.

}

{

BRIEFS

Afam Onyimadu, MakeUseOf: Privacy is a main reason people de-Google, but for me, switching from Google Search was about more than privacy. I didn't enjoy getting SEO-heavy results and listicles when I needed a simple debugging fix.

Yadullah Abidi, MakeUseOf: Windows' Power Options panel shows you a carefully curated list of power plans you can select to control how your laptop performs. But if you feel your laptop's performance isn't up to the hardware you paid for, there's a good chance Windows might be throttling it. And that's because there is one power plan Windows never shows you.

Graham King (Dark Coding), Graham King (Dark Coding): A software system is transparent when you can look at it and immediately understand what it is doing and how.

Jack Mitchell, MakeUseOf: In the past, I've sat down to watch something on Plex, only to be met with endless buffering — and it turns out I'm not alone. A quick check on Reddit revealed scores of posts dating back years seeking help with this very issue. It's especially annoying, as streaming apps are supposed to “just work” straight out of the box with minimal tweaking, regardless of the device or network conditions (although a few handy setup tips never hurt). However, as I found out, this lies at the root of the problem.

Stephen Clark, Ars Technica OpenForum: Jupiter's colossal storms generate lightning flashes at least 100 times more powerful than those on Earth, according to scientists analyzing data from NASA's Juno spacecraft.

Questions about the future of Juno and more than a dozen other robotic science missions began swirling nearly a year ago, when the Trump administration asked mission leaders to submit “closeout” plans for how to turn off their spacecraft. Ars first reported the news soon after the White House released a budget request that called for slashing NASA's science budget by nearly half.

[Read full article](#)

[Comments](#)

}

The Civic Ledger

Vol. 1, No. 015 · 2026-03-30

Curated and composed automatically.